LahoreLens: An AI-Driven Travel Companion Using Topic Modelling

Final Year Project – Mid Report (Phase I)

Session 2022-2026

A Final Year Project-I submitted in partial fulfillment of
the requirements for the Degree

of

BS in Computer Science

Department of Computer Science

University of the Punjab, Lahore

Project Details

Project ID (for office use)	
Type of project	[X] Traditional [ ] Industrial [ ] Continuing
Nature of project	[ ] Development [X] Research & Development
Sustainable Development Goals(SDGs)	[ ] Good Health and Well-Being [ ] Quality Education [X] Industry, Innovation, and Infrastructure [ ] Gender Equality [ ] Decent Work and Economic Growth [ ] Climate Action
Area of specialization	[X] Artificial Intelligence (AI) [X] Natural Language Processing (NLP) [X] Web Development

Project Group Members

Sr.#	Reg. #	Student Name	Email ID	Signature
(i)	BCSF22M047	M. Arham Nadeem	bcsf22m047@gmail.com	
(ii)	BCSF22M037	Ayesha Yaqoob	bcsf22m037@gmail.com	
(iii)	BCSF22M060	Maaz Mughal	bcsf22m060@gmail.com	
(iv)	BCSF22M007	Kashif Ali	bcsf22m007@gmail.com	

Declaration: The candidates confirm that the work submitted is their own and appropriate credit has been given where reference has been made to the work of others.

Plagiarism Free Certificate

This is to certify that, I am Muhammad Arham Nadeem Son of Muhammad Nadeem, group leader of FYP under registration no BSCS-F22-M047 at the Computer Science Department, Punjab University , Lahore . I declare that my FYP proposal is checked by my supervisor and the similarity index is 7% that is less than 20%, an acceptable limit by HEC. The report is attached herewith as Appendix A.

Date: _____ Name of Group Leader: Muhammad Arham Nadeem Signature: _____

Name of Supervisor: Dr. Fatima Sabir Co-Supervisor (if any):_____

Designation: _____ Designation: _____

Signature: _____ Signature: _____

HoD: _____ Signature: _____

ABSTRACT

The digital tourism landscape in Pakistan is rapidly expanding, yet existing travel platforms rely heavily on static databases that fail to capture the dynamic, real-time pulse of culturally rich cities like Lahore. Current applications lack the ability to process unstructured public sentiment, leaving a significant gap between curated listings and the authentic, trending experiences sought by modern travelers. This disconnect results in fragmented travel planning where users miss out on "hidden gems" and valid, real-time feedback available in public social media conversations. This project introduces "LahoreLens," an AI-driven travel companion that leverages a Microservices architecture to mine and analyse unstructured social media text from public APIs. By employing natural language processing (NLP) for Named Entity Recognition and Sentiment Analysis, alongside machine learning modelling on specific dataset, the system transforms raw public commentary into structured, trend-aware insights. The expected outcome is a functional web prototype that delivers personalized, socially validated travel recommendations, enabling users to explore Lahore's culinary and cultural events through real-time, data-driven intelligence rather than static directories.

Acknowledgement

We would like to express our deepest gratitude to our supervisor, Dr. Fatima Sabir, for her invaluable guidance, constant encouragement, and technical insights throughout the first phase of this project. Her expertise in Artificial Intelligence and Natural Language Processing has been instrumental in shaping the research direction and methodology of LahoreLens. We are also grateful to the Department of Computer Science (FCIT) for providing the resources and environment necessary to conduct this research and development work.

Furthermore, we would like to thank our families and friends for their unwavering support and patience during the intensive data collection and prototyping phases of this project. Their encouragement has been a vital source of motivation for us, as we move towards the completion of this project and our degrees.

Table of Contents

Chapter 1. Introduction 6
1.1 Introduction 6
1.2 Problem Statement 6
1.3 Proposed Solution 7
1.4 Main Objectives 7
1.5 Assumptions & Constraints 7
1.6 Project scope 8
1.7 Software Development Lifecycle Model 9
Chapter 2. Requirement Analysis 10
2.1 Literature Review 10
2.1.1 Brief Introduction 10
2.1.2 Tourism Recommendation Systems 10
2.1.3 NLP for Tourism Applications 11
2.1.4 Social Network Analysis for Personalization 11
2.1.5 Comparative Analysis of Travel Platforms 12
2.1.6 Research Gap Identification 14
2.1.7 Literature Review Summary Table 15
2.2 Stakeholders List 16
2.3 Requirements Elicitation 17
2.3.1 Functional Requirements 18
2.3.2 Non-functional Requirements 20
2.3.3 Requirements Traceability Matrix 24
2.4 Use Case Description 26
Chapter 3. System Design 29
3.1 Use Case Design 29
3.2 Activity Diagram 30
3.3 Sequence Diagram 31
3.4 Software Architecture Diagram 33
3.5 Class Diagram 34
3.6 Database Diagram 35
3.7 Collaboration Diagram 36
Chapter 4. System Testing 36
4.1 Test Cases Design 36
4.2 Unit / Integration / Acceptance Testing 39
Chapter 5. Implementation 40
5.1 Work Breakdown Structure (WBS) 41
5.2 Team Roles and Responsibilities 43
5.3 Tools and Technologies 44
5.4 Implementation Details 45
5.5 Screenshots of Prototype / System 45

5.6 Challenges During Implementation 48
Chapter 6. References 49

List of Tables

Table 2.1: Literature Review Summary 24
Table 2.2: Stakeholders List 24
Table 2.3: Functional Requirements 24
Table 2.4: Non-Functional Requirements 24
Table 2.5: Requirements Traceability Matrix 24
Table 4.1: System Test Cases and Validation Criteria 24
Table 5.1: Phase 1 Activity Log (Sept - Dec 2025 43
Table 5.3. Individual Tasks 43

List of Figures

Figure 3.1 LahoreLens Use Case Diagram 29
Figure 3.2 Main activity flow of LahoreLens 31
Figure 3.3 Sequence diagram 32
Figure 3.4 Software architecture of LahoreLens 33
Figure 3.5 Core domain class diagram for LahoreLens 34
Figure 3.6 MongoDB collection diagram for LahoreLens 35
Figure 3.7 Collaboration Diagram 36
Figure 4.1 Process Flowchart 40
Figure 5.1 Phase 1 Timeline 41
Figure 5.2 Gannt Chart 44

Chapter 1. Introduction

1.1 Introduction

The global tourism industry is currently undergoing a paradigm shift, moving away from static, one-size-fits-all travel guides toward dynamic, personalized digital experiences. In the context of Pakistan, the digital tourism sector is experiencing rapid expansion, with the market projected to exceed $5 billion by 2029, driven by a 25% annual increase in online bookings [1]. This growth is fuelled by a new generation of "digital-first" travellers who rely heavily on online ecosystems to plan their journeys. Lahore, as the cultural capital of Pakistan and its second-largest city, stands at the centre of this boom, attracting millions of domestic and international visitors annually who seek to explore its rich Mughal heritage, vibrant street food culture, and renowned culinary landscape.

However, the digital infrastructure supporting this influx is largely fragmented. While travellers have access to global platforms like Google Maps or TripAdvisor, these tools often rely on static databases and manual user reviews that can become outdated quickly. They fail to capture the "real-time pulse" of the city—the fleeting trends, the newly opened street food stalls, or the sudden popularity of a specific cultural event. The sheer volume of unstructured data available on social media platforms offers a solution to this, yet it remains an underutilized resource. There is a pressing need for intelligent systems that can mine this chaotic stream of public consciousness and translate it into structured, actionable travel guidance.

The primary motivation for "LahoreLens" arises from the limitations of existing tourism recommender systems. As noted by Xin et al. [2], the industry is shifting towards context-aware systems, yet many current applications still rely on basic collaborative filtering that ignores the rich context hidden in unstructured text. The significance of this project lies in its data-driven approach to solving the "cold start" and "relevance" problems in tourism recommendation systems. Unlike traditional apps that require manual data entry, our system automates the discovery of new places by analysing social media. Research by Li et al. [3] demonstrates that incorporating emotion-based preferences from online reviews significantly improves recommendation quality. Building on this foundation, LahoreLens utilizes publicly available data to extract public sentiment, offering a layer of social validation that static directories cannot match. Furthermore, Kumar et al. [4] have shown that sentiment analysis on platforms like Twitter is highly effective for tourism contexts; our project applies this specifically to the localized context of Lahore. This work is significant not only as a travel tool but as an R&D exploration into applying Microservices Architecture and Topic Modelling to solve real-world data mining challenges in the tourism sector.

1.2 Problem Statement

Despite Lahore's status as a premier tourist destination with rich cultural heritage and culinary diversity, travelers currently lack a unified digital platform that provides real-time, trend-aware, and culturally contextualized recommendations. We exceed existing solution on three merits: first, reliance on static, manually curated databases that lag behind actual trends and fail to capture the dynamic nature of urban tourism; second, an inability to process the vast amounts of unstructured social media data where authentic public opinions and real-time insights reside; and third, a lack of localized cultural intelligence that considers prayer times, halal preferences, and seasonal variations specific to Lahore. Consequently, users navigate a fragmented landscape of disparate applications and outdated blogs, often missing out on culturally significant events currently trending in the public sphere. This gap requires a system capable of using social media conversations, extracting useful information, and delivering recommendations that reflect Lahore's authentic tourism landscape.

1.3 Proposed Solution

To address these challenges, we propose "LahoreLens," an AI-driven web application designed to function as an intelligent travel companion for exploring Lahore. The solution is built upon a robust data pipeline that autonomously harvests unstructured text from public social media APIs (Twitter, YouTube comments) and comment histories from various platforms. Our approach employs a three-tier processing architecture: First, an NLP engine utilizing named entity recognition (NER) to automatically identify location names, events, and landmarks, coupled with sentiment analysis (SA) to classify public opinion as positive, negative, or neutral.

Second, a recommendation core implementing topic modeling (TM) using latent dirichlet allocation (LDA) to group locations by hidden themes (e.g., "Family-Friendly Cafés," "Historical Sunset Views") rather than rigid categories.

Third, a social network analysis (SNA) module that applies graph-based collaborative filtering to user search histories, identifying behavioral patterns and community preferences. The system is deployed using microservices architecture (MSA), with Node.js handling the application backend and Python microservices managing AI processing, ensuring independent scalability of data collection and analysis modules. A React.js frontend provides an intuitive user interface, while MongoDB stores structured insights extracted from social media. This hybrid approach combines real-time social sentiment with collaborative filtering to deliver culturally aware, trend-responsive recommendations unique to Lahore's tourism ecosystem.

1.4 Main Objectives

The specific objectives of this research and development project are:

- To design a scalable Data Collection Pipeline that aggregates unstructured text from multiple social media sources and public comment histories.
- To develop a custom NLP module for Named Entity Recognition and Sentiment Analysis with >70% accuracy.
- To implement topic modeling using latent dirichlet allocation (LDA) to discover hidden tourism themes within social media conversations.
- To apply social network analysis (SNA) to user search behaviors for creating a graph-based collaborative filtering model.
- To build an intelligent Recommendation Engine that ranks venues using real-time social sentiment, topic relevance, and trend velocity.
- To deliver a functional web prototype demonstrating the complete flow from raw text analysis to personalized user recommendations.

1.5 Assumptions & Constraints

Assumptions

- **API Accessibility:** It is assumed that public social media APIs (e.g., YouTube Data API) will remain accessible and their rate limits will accommodate the project's data collection needs.
- **Language Structure:** We assume that the unstructured text data collected will contain sufficient English or Roman Urdu content to be processable by standard machine learning libraries.
- **Data Quality:** It is assumed that the collected social media content contains genuine user opinions and experiences rather than being predominantly composed of spam or promotional bot content.

Constraints

- **Time Constraint:** The project must be completed within the academic session 2025-2026, necessitating a focus on core R&D functionality over polished commercial features.
- **Scope Constraint:** The prototype is a web application only; native mobile applications (Android/iOS) are currently outside the current scope of this project.
- **Data Constraint:** The system relies strictly on publicly available data; private groups or encrypted chats are out of bounds due to privacy policies and technical limitations.
- **Processing Power:** Training deep learning models for sentiment analysis (SA) requires substantial computational resources. We are constrained by the hardware available on standard development laptops, limiting the complexity of the models we can deploy.
- **API Rate Limits:** Social media APIs impose strict request limits (e.g., free tier restrictions), which constrain the volume and frequency of data collection we can perform daily.

1.6 Project scope

Included in Scope

- **Data Collection Pipeline:** Development of automated web scrapers and API integrations to collect unstructured text from Twitter, YouTube comments, and public review platforms.
- **NLP Processing Module:** Implementation of named entity recognition (NER), sentiment analysis (SA), and topic modeling (TM) using latent dirichlet allocation (LDA) to extract structured insights from social media text.
- **Recommendation Engine:** Development of a hybrid recommendation system combining content-based filtering (topic similarity), collaborative filtering via social network analysis (SNA), and trend-based ranking.
- **Backend Infrastructure:** Node.js-based application service with RESTful APIs for frontend communication and orchestration of Python-based natural language processing (NLP) microservices.
- **Database System:** MongoDB implementation for storing structured tourism data, user profiles, search histories, and processed social media insights.
- **Frontend Interface:** React.js web application providing user authentication, search functionality, recommendation display, and interactive maps for location visualization.
- **AI Chatbot:** Integration with Google Dialogflow for conversational assistance about Lahore's attractions, cuisine, and events.
- **Media Integration:** Text-to-image mapping for recommendations and URL linking to videos on their native platforms (YouTube).
- **Target Users:** Domestic and international tourists, Lahore residents seeking local experiences, and travel planners.

Excluded from Scope

- **Native Mobile Applications:** No iOS or Android native apps; the web application is mobile-responsive but not a dedicated mobile app.
- **Video Hosting:** Videos are linked via URLs to external platforms (YouTube); no video upload or streaming infrastructure is included.
- **Offline Functionality:** The application requires internet connectivity and does not support offline mode.
- **Advanced Urdu NLP:** While the system handles English content and basic Urdu transliterations, advanced Urdu natural language processing (NLP) is not included.

1.7 Software Development Lifecycle Model

This This project follows an **Agile Software Development Model** with iterative sprints and continuous integration. This methodology is particularly suitable for research and development (R&D) projects involving AI and data-driven systems where requirements often evolve based on model performance and data availability.

Justification for Agile

- **Iterative Refinement:** Our models and recommendation algorithms require continuous experimentation and tuning. Agile sprints allow us to develop initial prototypes, test them with real data, and improve accuracy progressively.
- **Risk Mitigation:** By developing components incrementally (Data Collection → NLP → Recommendation), technical challenges like API limits or data quality issues are identified and addressed early in the lifecycle.
- **Component Independence:** The microservices architecture aligns naturally with Agile, allowing independent development and testing of the data collection, backend, and frontend services.

Development Phases

The project lifecycle is structured into bi-weekly sprints, ensuring systematic progress across all SDLC phases:

- **Sprint 0 (Requirements & Architecture):** Define system requirements, design the microservices Architecture (MSA), select natural Language processing (NLP) models, and establish data sources.
- **Sprint 1-2 (Data Collection):** Develop the Python-based service to interface with social media APIs. Implement data cleaning and storage in MongoDB.
- **Sprint 3-4 (NLP Processing):** Implement named Entity recognition (NER), develop sentiment Analysis (SA) using Transformers, and integrate topic Modeling (TM).
- **Sprint 5-6 (Backend & Database):** Develop the Node.js application service, implement user authentication, and create RESTful APIs for frontend communication.
- **Sprint 7-8 (Recommendation Engine):** Implement social network analysis (SNA) for collaborative filtering and develop the hybrid ranking algorithm using term Frequency-Inverse Document Frequency (TF-IDF).
- **Sprint 9-10 (Frontend & Integration):** Develop the React.js user interface, integrate the AI chatbot, and connect all microservices.
- **Sprint 11-12 (Testing & Deployment):** Conduct accuracy testing of models, evaluate recommendation quality, perform user acceptance testing (UAT), and deploy to the cloud environment.

Continuous Processes

To ensure systematic progress, the team utilizes:

- **Version Control:** GitHub is used for code management with a branching strategy for parallel feature development.
- **Regular Reviews:** Weekly stand-ups and sprint reviews with the supervisor ensuring the project aligns with academic research object.

Chapter 2. Requirement Analysis

2.1 Literature Review

2.1.1 Brief Introduction

The evolution of intelligent travel guidance has been accelerated by breakthroughs in machine learning and the massive proliferation of user-generated content. This literature review explores four critical pillars: recommendation algorithms, **text mining strategies** for unstructured data, social network analysis for personalization, and the current landscape of commercial travel applications. By synthesizing findings from recent academic research and industry implementations, this review establishes the theoretical foundation for our approach while identifying significant limitations in existing platforms—specifically the lack of real-time social sensing and localized cultural context. The analysis reveals that while global solutions are sophisticated, there is a distinct absence of integrated systems that combine text extraction and opinion mining to provide context-aware tourism guidance within Pakistan.

2.1.2 Tourism Recommendation Systems

Tourism recommendation systems have evolved from simple directory listings to sophisticated AI-driven platforms capable of personalized suggestions. Xin et al. [1] provide a comprehensive review of trends in tourism recommendation systems, categorizing approaches into content-based filtering, collaborative filtering, and hybrid models. Content-based filtering recommends locations based on similarities to users' past preferences, while collaborative filtering leverages patterns from similar users to generate suggestions. The authors demonstrate that modern systems increasingly adopt hybrid approaches to overcome the limitations of individual methods, particularly the "cold start" problem where new users or locations have insufficient data.

Kzaz et al. [5] further classify tourism recommender systems by examining the underlying recommendation techniques, including knowledge-based, demographic-based, and context-aware approaches. Their analysis reveals that context-aware systems, which consider factors such as time, location, weather, and user mood, significantly outperform traditional methods in user satisfaction scores. However, they note that most context-aware systems rely on explicitly provided user preferences rather than implicitly derived insights from user behavior or social media activity.

The findings of Berka and Plößnig [6] emphasize the importance of combining multiple filtering techniques to handle the complexity and multifaceted nature of tourism data. Their design framework for tourism recommender systems highlights that travel decisions are influenced by numerous factors—budget, travel companions, cultural interests, dietary restrictions—making single-approach systems inadequate. They advocate for multi-criteria decision-making models that can weigh diverse user preferences simultaneously.

Li et al. [3] introduce a proactive tourism recommender system that incorporates emotion-based preferences extracted from online reviews. Their research demonstrates that sentiment analysis of user-generated content significantly improves recommendation quality, with their system achieving a 23% improvement in user satisfaction compared to baseline collaborative filtering approaches. This work provides strong empirical support for our project's core hypothesis: that real-time public sentiment from social media contains valuable signals for tourism recommendations. However, Li's system processes formal reviews rather than the more dynamic, unstructured conversations found on platforms like Twitter.

2.1.3 NLP for Tourism Applications

Natural Language Processing (NLP) techniques have become essential for extracting structured insights from the vast amounts of unstructured text generated by travelers. This section examines three key NLP domains relevant to LahoreLens: sentiment analysis, named entity recognition, and topic modeling.

Sentiment Analysis for Tourism

Demonstrated by Kumar et al. [4], the efficacy of hybrid deep learning models for tourism sentiment analysis on Twitter. By combining Convolutional Neural Networks (CNN) for feature extraction with Bidirectional LSTM (BiLSTM) for sequence modeling, they achieved 92.3% accuracy in classifying tourist sentiment. Their findings highlight that even brief, informal social media text contains sufficient signal for accurate classification and that hybrid architectures outperform single-model approaches. Crucially, the study notes that real-time analysis allows for faster detection of emerging issues compared to traditional reviews. However, their work is limited to destination-level sentiment, lacking the venue-level granularity required for city exploration apps like LahoreLens.

Named Entity Recognition for Location Extraction

For location extraction, Liu, Wang, and Sun [7] propose a BERT-based approach specifically optimized for social media text. Their model addresses the challenge of informal references and abbreviations common in user-generated content, achieving an 87.4% F1-score, a significant improvement over general-purpose systems. This research is highly relevant to LahoreLens, as Pakistani tourism involves similar complexities regarding colloquial names and romanized Urdu transliterations (e.g., "Food Street" instead of "Gawalmandi"). The study establishes that domain-specific fine-tuning is essential for accurate extraction, though its reliance on Chinese datasets highlights the need for Pakistan-specific training data.

Topic Modeling for Theme Discovery

Topic modeling moves beyond rigid keyword categorization to uncover latent themes in travel experiences. Research by Blei, Ng, and Jordan [8] on latent dirichlet allocation (LDA) enables the automatic discovery of coherent topics like "family-friendly weekends" or "late-night food culture" directly from user discussions. Studies confirm that this thematic organization leads to more intuitive browsing and higher user engagement compared to static taxonomies. However, existing applications often rely on structured blogs rather than noisy social media text and operate in batch modes, which limits their ability to detect real-time trends, a gap LahoreLens aims to address.

NLP Challenges in Multilingual Contexts

Processing tourism text in Pakistan presents unique challenges due to code-mixing and romanized Urdu. Riaz [9] highlights that standard English NLP tools perform poorly on South Asian social media text, with accuracy dropping below 60% when Urdu terms are present. This necessitates a strategic trade-off for LahoreLens: balancing the broad coverage of English content with the complexity of training custom models for romanized Urdu processing.

2.1.4 Social Network Analysis for Personalization

Social Network Analysis (SNA) techniques enable recommendation systems to leverage relationships and behavioral patterns among users for improved personalization.

Hassan et al. [10] propose a hybrid recommender system for the tourism industry that combines evolutionary Apriori algorithm with K-means clustering to identify user communities with similar travel preferences. Their approach models users as nodes in a graph, with edges representing similarity in search behavior, booking patterns, or review activities.

The research demonstrates that graph-based collaborative filtering outperforms traditional user-item matrix approaches when data is sparse. By identifying user clusters through community detection algorithms, the system can recommend locations popular within a user's peer group even when the user has provided minimal explicit preferences. Hassan's model achieves 78.3% precision in predicting user interest in new destinations, representing a 12% improvement over baseline collaborative filtering methods.

Graph-based models offer advantages for tourism applications. First, they naturally handle cold-start problems by leveraging transitive relationships—if user A and user B share preferences, and user B and user C share preferences, the system can recommend to user C items liked by user A. Second, they enable community-based trend detection, identifying locations that are suddenly gaining popularity within specific user segments. Third, they support contextual personalization by analyzing not just what users prefer, but when and with whom they travel.

However, existing SNA applications in tourism typically require explicit social connections (friends, followers) or registered user accounts to construct social graphs. LahoreLens aims to extend this approach by inferring implicit similarity from anonymous search behaviors and location visit patterns, enabling personalization without requiring user registration or social network access.

2.1.5 Comparative Analysis of Travel Platforms

To understand the current state of digital tourism tools and identify gaps that LahoreLens addresses, we conducted a technical analysis of ten major travel platforms spanning global, regional, and local markets. This analysis examines their architecture, features, data sources, and limitations.

Global Platforms: TripAdvisor, Google Maps, and Airbnb

TripAdvisor, the world's largest travel platform with over 1 billion reviews, operates on microservices architecture using Java-based services, Apache Cassandra for distributed data storage, and Elasticsearch for search functionality. The platform excels at aggregating user-generated reviews but relies on static, manually curated databases. Recommendations are driven primarily by popularity ranking—sorting venues by review count and average rating—rather than personalized algorithms that consider individual preferences or real-time trends. Our analysis reveals that TripAdvisor's content becomes outdated between user review cycles, typically lagging in actual conditions by weeks or months.

Google Maps leverages unparalleled data integration across Google's ecosystem, processing real-time location data from Android devices to update traffic conditions and popular times for venues. Its recommendation engine uses collaborative filtering based on search history and location visits. However, Google's approach lacks deep semantic understanding of user intent. A search for "romantic restaurants" returns venues tagged with that keyword rather than discovering locations organically discussed as romantic in social media conversations. Additionally, Google's global architecture does not embed localized cultural intelligence—it cannot prioritize halal restaurants for users during Ramadan or suggest family-friendly venues that accommodate prayer times.

Airbnb has pioneered personalization in accommodation booking through its recommendation engine, which analyzes user behavior, search patterns, and booking history using deep learning models.

The platform's strength lies in matching travelers with unique stays based on nuanced preferences. However, Airbnb's focus on accommodation limits its utility as a comprehensive city exploration tool. It does not integrate real-time social media sentiment, provide dynamic attraction recommendations, or offer culturally contextualized guidance for destinations like Lahore.

Regional Platforms: FindMyAdventure and Bookme

Pakistan-based platforms like FindMyAdventure and Bookme represent the regional digital tourism landscape. FindMyAdventure focuses on curated tour packages for adventure tourism in northern Pakistan, operating a marketplace model that connects users with tour operators. The platform uses basic filtering (destination, duration, price) rather than intelligent recommendations. Content is vendor-supplied and updated quarterly at best, providing no mechanism for capturing real-time traveler sentiment or emerging destinations.

Bookme specializes in ticketing services—cinema, events, hotels, buses—with a straightforward inventory management system built on Node.js and MySQL. Its recommendation capability is limited to displaying popular events or "trending" categories determined by booking volume. The platform does not employ natural language processing (NLP), machine learning, or social media analysis. User experience relies on manual search rather than intelligent discovery.

Local Platforms: Lahore Government Portal and Rasta

The official Lahore tourism portal (lahore.punjab.gov.pk) exemplifies the digital gap our project addresses. Built on outdated web technologies (basic Hypertext Markup Language (HTML) and Cascading Style Sheets (CSS) without modern frameworks), the site provides static information about major landmarks but lacks interactivity, personalization, or real-time updates. Content is government-authored rather than reflecting authentic traveler experiences. The site has no mobile responsiveness, search functionality, or integration with maps or booking systems.

Rasta, developed by Punjab Information Technology Board, focuses on traffic management and driving license services rather than tourism. While it demonstrates that local government can build functional mobile applications, its single-purpose design and lack of tourism features highlight the absence of comprehensive digital tourism tools for Lahore residents and visitors.

Gap Analysis Summary

Our comparative analysis reveals five critical gaps that LahoreLens addresses:

First, no existing platform actively mines social media for real-time tourism insights. All platforms rely on user reviews (posted hours to days after experiences) or static databases (updated monthly or quarterly). This prevents detection of sudden trends, temporary closures, or emerging venues.

Second, current systems lack semantic understanding of unstructured text. Keyword tagging and simple text search cannot capture the nuanced themes that topic modeling via latent dirichlet allocation (LDA) reveals such as "budget-friendly family outings" or "Tweetable sunset spots."

Third, personalization is either absent (government portals, regional platforms) or based on explicit user data and account history (global platforms). No system infers preferences from anonymous behavioral patterns using social network analysis (SNA).

Fourth, cultural and contextual intelligence is uniformly lacking. Global platforms apply generic algorithms worldwide, missing localized factors like halal preferences, prayer times, seasonal festivals, and neighborhood safety that significantly influence tourism decisions in South Asian contexts.

Fifth, real-time trend detection is non-existent. Popularity rankings reflect cumulative historical data rather than current momentum, preventing users from discovering newly popular venues or avoiding locations experiencing recent issues.

2.1.6 Research Gap Identification

The literature review and platform analysis collectively reveal a significant research gap at the intersection of real-time social media analysis, tourism recommendation systems, and localized cultural intelligence. While substantial research exists on tourism recommender systems and text mining, their integration, specifically for city-level, culturally aware tourism guidance remains largely unexplored.

Existing tourism recommendation research predominantly focuses on global platforms, hotel/flight booking, or general destination selection at country or region granularity. Few studies address the challenge of urban exploration within a single city, where recommendations must span diverse categories (dining, heritage sites, entertainment, shopping) while remaining sensitive to cultural context and real-time conditions.

Current NLP applications in tourism typically process formal reviews on dedicated platforms (TripAdvisor, Booking.com) rather than the more dynamic, unstructured, and conversational content found on social media. Research by Kumar et al. [4] demonstrates Twitter sentiment analysis for tourism but focuses on destination-level sentiment rather than venue-specific insights. Topic modeling studies apply latent dirichlet allocation (LDA) to travel blogs and reviews, but do not address real-time processing of social media streams or localized theme discovery.

The application of social network analysis (SNA) in tourism contexts, as explored by Hassan et al. [10], requires explicit social connections or registered user accounts. No research examines implicit user similarity from anonymous search patterns and behavioral data crucial capability for privacy-conscious systems that do not mandate user registration.

Most critically, existing systems universally lack localized cultural intelligence. Research on tourism recommendation systems overwhelmingly originates from Western academic institutions and focuses on Western tourism contexts. The unique requirements of South Asian tourism—halal certification, prayer time considerations, gender-segregated spaces, seasonal festival awareness—receive minimal attention in academic literature and zero consideration in commercial platform designs.

LahoreLens addresses this gap by proposing an integrated system that: (1) continuously mines social media for real-time venue-level insights (2) discovers latent themes through topic modeling to enable intuitive exploration beyond rigid categories, (3) personalizes recommendations via graph-based Social Network Analysis (SNA) without requiring user registration, (4) embeds Lahore-specific cultural context into the data model and recommendation algorithms, and (5) operates at city granularity with detail rather than simple overview-level generalization.

This research gap is not merely technical but has practical significance. Lahore attracts millions of domestic and international visitors annually, yet travelers navigate a fragmented landscape of outdated government portals, generic global platforms, and word-of-mouth recommendations.

The absence of an intelligent, culturally aware, real-time tourism companion represents a missed opportunity to enhance visitor experiences, support local businesses, and promote Pakistan's tourism industry through modern digital infrastructure.

2.1.7 Literature Review Summary Table

The following table summarizes key research studies examined in this literature review, presenting their methodologies, findings, and relevance to the LahoreLens project.

Table 2.1: Literature Review Summary

Reference	**Research Focus**	**Methodology**	**Key Findings**	**Relevance to LahoreLens**
Xin et al.	Tourism recommendation system trends	Systematic review of content-based, collaborative, and hybrid filtering	Hybrid models outperform single approaches; context-aware systems show highest user satisfaction	Validates hybrid recommendation approach combining content-based (topic similarity) and collaborative (SNA) filtering
Kzaz et al.	Tourism recommender system approaches	Classification and comparison of knowledge-based, demographic, and context-aware methods	Context-aware systems achieve 23% higher user satisfaction than traditional methods	Supports design decision to incorporate temporal and cultural context (prayer times, festivals) into recommendations
Li et al.	Emotion-based tourism recommendations	Sentiment Analysis (SA) on online reviews using deep learning	23% improvement in recommendation quality when incorporating sentiment features	Provides empirical justification for our core hypothesis that social media sentiment improves recommendations
Kumar et al.	Twitter sentiment analysis for tourism	Hybrid Convolutional Neural Network (CNN) and Bidirectional Long Short-Term Memory model	92.3% accuracy in tourism sentiment classification; real-time analysis detects emerging trends faster than review systems	Demonstrates feasibility of Twitter-based sentiment analysis; validates deep learning approach for tourism text
Liu et al.	Named Entity Recognition (NER) for location-based services	BERT fine-tuning on social media location mentions	87.4% F1-score, 15% improvement over general NER models through domain-specific training	Highlights need for Pakistan-specific NER training to handle Urdu transliterations and colloquial place names

Reference	Research Focus	Methodology	Key Findings	Relevance to LahoreLens
Blei et al.	Latent Dirichlet Allocation (LDA) topic modeling	Probabilistic generative model for topic discovery	LDA effectively identifies latent themes in unstructured text without supervision	Theoretical foundation for using LDA to discover hidden themes ("family-friendly," "late-night food") in social media
Hassan et al.	Social Network Analysis (SNA) for tourism recommendations	Graph-based collaborative filtering with K-means clustering	78.3% precision, 12% improvement over baseline; handles cold-start through transitive relationships	Validates graph-based approach for personalization; informs our SNA design for user behavior clustering

2.2 Stakeholders List

The following table identifies the key individuals and groups involved in or affected by the LahoreLens project. Understanding these stakeholders ensures that the system is designed to meet diverse needs, from user usability to academic compliance.

Table 2.2: Stakeholders List

Stakeholder Category	Stakeholder Name	Role & Responsibility	Key Expectation
Primary Users	Domestic & International Tourists	The primary end-users who will interact with the web application to search for locations and view recommendations.	Expect accurate, real-time, and culturally relevant recommendations (e.g., halal food, safe areas).
Primary Users	Lahore Residents	Secondary users who utilize the system to discover trending dining spots, events, or "hidden gems" within their own city.	Interested in real-time "trending" updates and avoiding overcrowded or closed venues.
Project Team	Development Team (Students)	Responsible for the end-to-end design, implementation, testing, and documentation of the system.	Goal is to successfully demonstrate the R&D objectives, validate the NLP models, and complete the degree requirements.

Stakeholder Category	Stakeholder Name	Role & Responsibility	Key Expectation
Academic	Project Supervisor	Provides technical guidance, reviews progress, and ensures the project meets university research standards.	Expects academic rigor, adherence to the timeline, and technical depth in the AI/NLP implementation.
Academic	Evaluation Committee	Internal and external examiners who assess the final project during the defense (Viva).	Interested in the functionality of the prototype, the validity of the research gap, and the performance metrics of the models.
Data Providers	Social Media Platforms (Twitter, YouTube, Google)	External entities that provide the data infrastructure (APIs) required for the system to function.	Their "interest" is strictly technical; the system must comply with their API usage policies and rate limits.
Beneficiaries	Local Tourism Ecosystem	Local restaurants, heritage sites, and small businesses in Lahore that are recommended by the system.	They benefit indirectly through increased visibility and footfall generated by the app's recommendations.

2.3 Requirements Elicitation

Requirements elicitation for LahoreLens was conducted through comprehensive domain analysis and comparative system evaluation. Through a process focused on synthesizing functional needs from three primary sources: (1) a rigorous review of academic literature to identify gaps in current tourism algorithms, (2) a feature-gap analysis of leading global and local travel platforms, (3) technical consultation with the project supervisor to establish feasibility boundaries.

To ensure user-centricity, we employed User Persona Modeling, constructing detailed profiles for domestic tourists, international visitors, and local food enthusiasts to simulate diverse use cases. The elicitation process followed an iterative cycle: initial requirements were drafted based on the identified research gaps, validated against the project's technical constraints (API limits, processing power), and prioritized using the MoSCoW method (Must have, Should have, Could have, Won't have). This approach ensured that the requirements are grounded in technical reality while addressing the critical deficiencies found in existing solutions.

We further researched and added all our findings in preparation of our Literature Rivew.

2.3.1 Functional Requirements

Functional requirements define the specific behaviors, features, and services that the LahoreLens system must provide. Each requirement is assigned a unique identifier.

Table 2.3: Functional Requirements

Requirement ID	Requirement Description	Priority	Source
FR-01	The system shall collect unstructured text data from Twitter Application Programming Interface (API) and public comment histories on a daily basis	Must Have	Project Objective 1
FR-02	The system shall preprocess collected text data by removing stop words, special characters, and performing tokenization	Must Have	Technical Requirement
FR-03	The system shall perform Named Entity Recognition (NER) to extract location names, restaurant names, and event names from social media text	Must Have	Project Objective 2
FR-04	The system shall classify sentiment of extracted entities as positive, negative, or neutral using Sentiment Analysis (SA) models	Must Have	Project Objective 2
FR-05	The system shall apply Latent Dirichlet Allocation (LDA) topic modelling to discover hidden themes in tourism-related conversations	Must Have	Project Objective 2
FR-06	The system shall store processed data including entities, sentiment scores, and topic distributions in MongoDB database	Must Have	Technical Requirement
FR-07	The system shall build a user search behaviour graph using Social Network Analysis (SNA) techniques to identify similar users	Must Have	Project Objective 3
FR-08	The system shall generate personalized location recommendations by combining topic similarity (content-based) and user similarity (collaborative)	Must Have	Project Objective 4
FR-09	The system shall rank recommendations based on real-time sentiment trend velocity and cumulative popularity	Must Have	Project Objective 4

Requirement ID	Requirement Description	Priority	Source
FR-10	The system shall provide a search interface allowing users to query by location type, cuisine, or activity	Must Have	User Requirement
FR-11	The system shall display recommendation results with location name, category, sentiment score, trending indicator, and brief description	Must Have	User Requirement
FR-12	The system shall integrate Google Maps API to display location markers and enable navigation	Should Have	User Requirement
FR-13	The system shall implement user authentication using Firebase Authentication for saving preferences and search history	Should Have	User Requirement
FR-14	The system shall integrate conversational chatbot using Google Dialogflow for natural language queries about Lahore	Should Have	Project Objective 5
FR-15	The system shall fetch location images from third-party APIs (Google Places) for visual representation	Should Have	User Experience
FR-16	The system shall provide filtering options by price range, distance, operating hours, and halal certification	Should Have	Cultural Context
FR-17	The system shall display prayer times and highlight nearby mosques when relevant to user queries	Could Have	Cultural Context
FR-18	The system shall track trending locations and display "trending now" badge for venues with increasing positive sentiment	Could Have	Feature Enhancement
FR-19	The system shall generate automated summaries of user reviews for each location using extractive summarization	Could Have	Feature Enhancement
FR-20	The system shall allow users to save favorite locations and create custom trip itineraries	Could Have	User Experience

2.3.2 Non-functional Requirements

Non-functional requirements specify the quality attributes, performance criteria, and constraints that the system must satisfy. These requirements define how the system performs its functions rather than what functions it performs.

Table 2.4: Non-Functional Requirements

Requirement ID	**Requirement Description**	**Priority**	**Metric**
Performance			
NFR-01	The system shall process Named Entity Recognition (NER) and Sentiment Analysis (SA) for social media data within 24 hours of collection.	Must Have	Daily batch processing completion within 24-hour window.
NFR-02	The system shall return search results and recommendations to users within 2 seconds for 95% of queries.	Must Have	Response time ≤ 2 seconds (p95).
NFR-03	The system shall support concurrent usage by at least 100 simultaneous users without performance degradation.	Should Have	100 concurrent users with response time ≤ 3 seconds.
NFR-04	The Natural Language Processing (NLP) module shall achieve minimum 70% accuracy on Named Entity Recognition (NER) for Lahore locations.	Must Have	F1-score ≥ 0.70 on test dataset.
NFR-05	The Sentiment Analysis (SA) model shall achieve minimum 70% accuracy in classifying text sentiment.	Must Have	Accuracy ≥ 0.70 on validation set.

Requirement ID	Requirement Description	Priority	Metric
Reliability			
NFR-06	The system shall maintain 95% uptime during testing phase (excluding scheduled maintenance).	Should Have	Monthly uptime ≥ 95%.
NFR-07	The system shall implement error handling for Application Programming Interface (API) failures and gracefully degrade functionality.	Must Have	No system crashes due to external API failures.
NFR-08	The data collection service shall retry failed Twitter API requests up to 3 times with exponential backoff.	Must Have	Successful completion or logged failure after 3 retries.
Usability			
NFR-09	The web interface shall be responsive and function correctly on desktop (1920x1080), tablet (768x1024), and mobile (375x667) screens.	Must Have	Verified across 3 device categories.
NFR-10	The system shall provide intuitive navigation requiring no more than 3 clicks to reach any major feature.	Should Have	Maximum 3-click depth for core features.
NFR-11	The system shall display error messages in user-friendly language (avoiding technical jargon) with clear guidance for resolution.	Must Have	User testing validation.

Requirement ID	Requirement Description	Priority	Metric
NFR-12	The search interface shall support both English and basic Urdu transliterated queries.	Could Have	Query success rate for romanized Urdu ≥ 60%.
Security			
NFR-13	The system shall encrypt all user authentication data using industry-standard encryption (Advanced Encryption Standard (AES) 256-bit).	Must Have	All passwords hashed with bcrypt/Firebase Auth.
NFR-14	The system shall implement Hypertext Transfer Protocol Secure (HTTPS) for all client-server communication.	Must Have	All endpoints use Transport Layer Security (TLS) 1.2+.
NFR-15	The system shall not store personally identifiable information from social media beyond entity mentions and sentiment.	Must Have	Data privacy compliance verification.
NFR-16	The system shall implement rate limiting on public Application Programming Interfaces (APIs) to prevent abuse (100 requests per minute per IP).	Should Have	Rate limit enforcement on all public endpoints.
Maintainability			
NFR-17	The system architecture shall follow Microservices Architecture (MSA) with independent deployment of data collection, Natural Language Processing (NLP), and backend services.	Must Have	Three independently deployable services.

Requirement ID	Requirement Description	Priority	Metric
NFR-18	The codebase shall maintain minimum 70% code documentation coverage for all functions and classes.	Should Have	Documentation coverage ≥ 70% measured by tools.
NFR-19	The system shall use version control (Git) with documented commit messages following conventional commit standards.	Must Have	All code in Git repository with semantic commits.
NFR-20	The MongoDB database shall support horizontal scaling to handle growing data volume.	Should Have	Database design allows sharding/replication.
Compatibility			
NFR-21	The web application shall be compatible with Chrome (v100+), Firefox (v95+), Safari (v15+), and Edge (v100+) browsers.	Must Have	Functional testing across 4 browsers.
NFR-22	The system shall function correctly with JavaScript enabled and gracefully degrade with JavaScript disabled.	Should Have	Core features accessible without JavaScript.
NFR-23	The Natural Language Processing (NLP) models shall be compatible with Python 3.9+ runtime environment.	Must Have	Verified on Python 3.9 and 3.10.

2.3.3 Requirements Traceability Matrix

The Requirements Traceability Matrix establishes traceability between project objectives, functional requirements, non-functional requirements, and system components. This ensures that every requirement can be traced back to a business need and forward to implementation components, facilitating change management and verification.

Table 2.5: Requirements Traceability Matrix

Requirement ID	**Requirement Type**	**Description Summary**	**Maps to Objective**	**Implements in Component**	**Verification Method**	**Priority**
FR-01	Functional	Data collection from Twitter API	Objective 1	Data Collection Service (Python)	Integration test, Daily log verification	Must Have
FR-02	Functional	Text preprocessing	Objective 2	NLP Service (Python)	Unit test with sample data	Must Have
FR-03	Functional	Named Entity Recognition (NER)	Objective 2	NLP Service (spaCy/BERT)	Accuracy test on validation set	Must Have
FR-04	Functional	Sentiment Analysis (SA)	Objective 2	NLP Service (Transformers)	F1-score on test dataset	Must Have
FR-05	Functional	Topic modeling with Latent Dirichlet Allocation (LDA)	Objective 2	NLP Service (scikit-learn)	Coherence score measurement	Must Have
FR-06	Functional	Data storage in MongoDB	Technical	Backend Service (Node.js)	Database connectivity test	Must Have
FR-07	Functional	Social Network Analysis (SNA) graph construction	Objective 3	Backend Service (Network X)	Graph metrics validation	Must Have
FR-08	Functional	Hybrid recommendation generation	Objective 4	Recommendation Engine (Python)	Precision@K evaluation	Must Have
FR-09	Functional	Trend-based ranking	Objective 4	Recommendation Engine	Manual verification of trending items	Must Have
FR-10	Functional	Search interface	Objective 5	Frontend (React.js)	User acceptance testing	Must Have

Requirement ID	Requirement Type	Description Summary	Maps to Objective	Implements in Component	Verification Method	Priority
FR-11	Functional	Results display	Objective 5	Frontend (React.js)	UI verification	Must Have
FR-12	Functional	Google Maps integration	User Experience	Frontend + Google Maps API	Integration test	Should Have
FR-13	Functional	User authentication	Security	Backend + Firebase Auth	Authentication flow test	Should Have
FR-14	Functional	Chatbot integration	Objective 5	Dialogflow + Backend	Conversation testing	Should Have
FR-15	Functional	Image fetching	User Experience	Backend + Google Places API	API response validation	Should Have
FR-16	Functional	Filtering options	Cultural Context	Frontend + Backend	Filter functionality test	Should Have
NFR-01	Performance	Daily batch processing within 24 hours	Objective 1	Data Collection + NLP Services	Monitoring logs	Must Have
NFR-02	Performance	2-second response time (p95)	Usability	Full Stack	Load testing with Apache JMeter	Must Have
NFR-03	Performance	100 concurrent users	Scalability	Backend + Database	Stress testing	Should Have
NFR-04	Performance	70% NER accuracy	Objective 2	NLP Service	Test dataset evaluation	Must Have
NFR-05	Performance	70% SA accuracy	Objective 2	NLP Service	Validation set metrics	Must Have
NFR-06	Reliability	95% uptime	System Quality	Infrastructure (AWS)	Uptime monitoring	Should Have
NFR-13	Security	Encrypted authentication	Security	Firebase Auth	Security audit	Must Have
NFR-14	Security	HTTPS for all communication	Security	Backend (Express.js)	SSL certificate verification	Must Have
NFR-17	Maintainability	Microservices architecture	Technical	System Architecture	Architecture review	Must Have
NFR-21	Compatibility	Cross-browser support	Usability	Frontend	Browser compatibility testing	Must Have

2.4 Use Case Description

Table 2.6 UC-01 Search for Location Recommendations

Field	Description
Use Case ID	**UC-01**
Use Case Name	**Search for Location Recommendations**
Primary Actor	Tourist/User (Domestic or International Visitor)
Secondary Actors	Recommendation Engine, MongoDB Database, Google Maps API
Preconditions	1. User has accessed the LahoreLens web application. 2. System database contains processed social media data (past 7 days). 3. Internet connectivity is active.
Main Flow	1. **User** navigates to the homepage and enters a query (e.g., "family restaurants"). 2. **System** parses the query to extract location type, context keywords, and geographic constraint. 3. **System** retrieves user's search history (if authenticated) or session data. 4. **System** queries MongoDB for locations with Positive Sentiment (>0.6) and relevant Topic Distribution (LDA). 5. **System** ranks results using hybrid algorithm (Content + Collaborative + Trend Velocity). 6. **System** displays top 10 recommendations with Sentiment Indicator and Trending Badge. 7. **User** clicks on a recommendation card. 8. **System** displays detailed view including Map, Topic Tags, and option to get directions.
Alternate Flows	**A1: No Results Found** 1. System relaxes constraints (expands radius/reduces threshold). 2. System displays "No exact matches found" with broader results. **B1: System Error** 1. MongoDB query fails. 2. System logs error and displays "Retry" button to user.
Post-conditions	**Success:** User receives relevant recommendations; search query is logged for SNA. **Failure:** Error is logged; system remains stable without data corruption.

Table 2.7 UC-02 Social Media Data Ingestion and Analysis

Field	**Description**
Use Case ID	**UC-02**
Use Case Name	**System Processes Daily Social Media Data**
Primary Actor	System Scheduler (Automated)
Secondary Actors	Data Collection Service, NLP Service, MongoDB, Twitter API
Preconditions	1. Twitter API credentials are valid. 2. NLP Microservice is operational. 3. System scheduler is configured for daily execution (2:00 AM).
Main Flow	1. **Scheduler** triggers Data Collection Service at 2:00 AM. 2. **Service** requests data from Twitter API (Keywords: "Lahore", "Food", "Visit"; Radius: 50km). 3. **Service** cleans data (removes duplicates/spam) and stores raw tweets in MongoDB. 4. **NLP Service** retrieves pending tweets and applies preprocessing (Tokenization, Stop-words). 5. **NLP Service** runs **NER** to extract Location/Event entities. 6. **NLP Service** runs **Sentiment Analysis** to classify entities (Pos/Neg/Neu). 7. **NLP Service** runs **LDA Topic Modeling** to assign thematic categories. 8. **System** stores Processed Insights (Entity, Score, Topic) in MongoDB. 9. **Scheduler** marks job as complete.
Alternate Flows	**A1: API Rate Limit Exceeded** 1. Service detects 429 Error. 2. Service logs warning and implements exponential backoff (Wait 15 mins). **B1: Low Confidence Scores** 1. NER confidence < 0.5. 2. Entities flagged for manual review; processing continues for high-confidence data.
Post-conditions	**Success:** Fresh social media insights are available for the Recommendation Engine. **Failure:** Administrator alerted; partial data saved; no data corruption.

Table 2.8 UC-03 Conversation Assistance through Chatbots

Field	**Description**
Use Case ID	**UC-03**
Use Case Name	**Interact with Artificial Intelligence (AI) Chatbot**
Primary Actor	Tourist/User
Secondary Actors	Google Dialogflow, Backend Service, MongoDB Database
Preconditions	1. Chatbot integration with Google Dialogflow is active. 2. Backend service can process webhook requests. 3. Database contains location event data.
Main Flow	1. **User** clicks the chatbot icon to open the overlay. 2. **User** types a natural language query (e.g., "Best historical places this weekend?"). 3. **System** sends query to Dialogflow NLP engine to identify Intent and Entities. 4. **Backend** constructs database query based on extracted parameters (e.g., Category="Historical", Time="Weekend"). 5. **System** retrieves top 5 matching locations from MongoDB. 6. **System** formats response as conversational text with clickable cards. 7. **User** reads response and either clicks a card or asks a follow-up.
Alternate Flows	**A1: Query Not Understood** 1. Dialogflow confidence score < 0.5. 2. System responds with clarification request and example buttons. **B1: Service Unavailable** 1. Dialogflow API unreachable. 2. System displays "Assistant temporarily unavailable" message.
Post-conditions	**Success:** User receives conversational response; context is maintained for multi-turn interaction. **Failure:** Error handled gracefully; user guided to manual search.

Chapter 3. System Design

This chapter explains how LahoreLens is structured from a system perspective, covering user interactions, internal workflows, and the Microservices Architecture that supports data collection, Natural Language Processing NLP, recommendation generation, and web delivery.

3.1 Use Case Design

LahoreLens has three main actors. These include User, System Scheduler, and AI Chatbot service. The use cases include searching for recommendations, consumption of daily social media data, and use of the chatbot service.

• UC-01: Search for Lahore Recommendations

• UC-02: Social Media Data Ingestion and Analysis

• UC-03: Conversation Assistance through Chatbots

Every case of usage defines actors, preconditions, main flow, alternate flows, and post-conditions to ensure functional requirements are used to describe user system interactions functions.

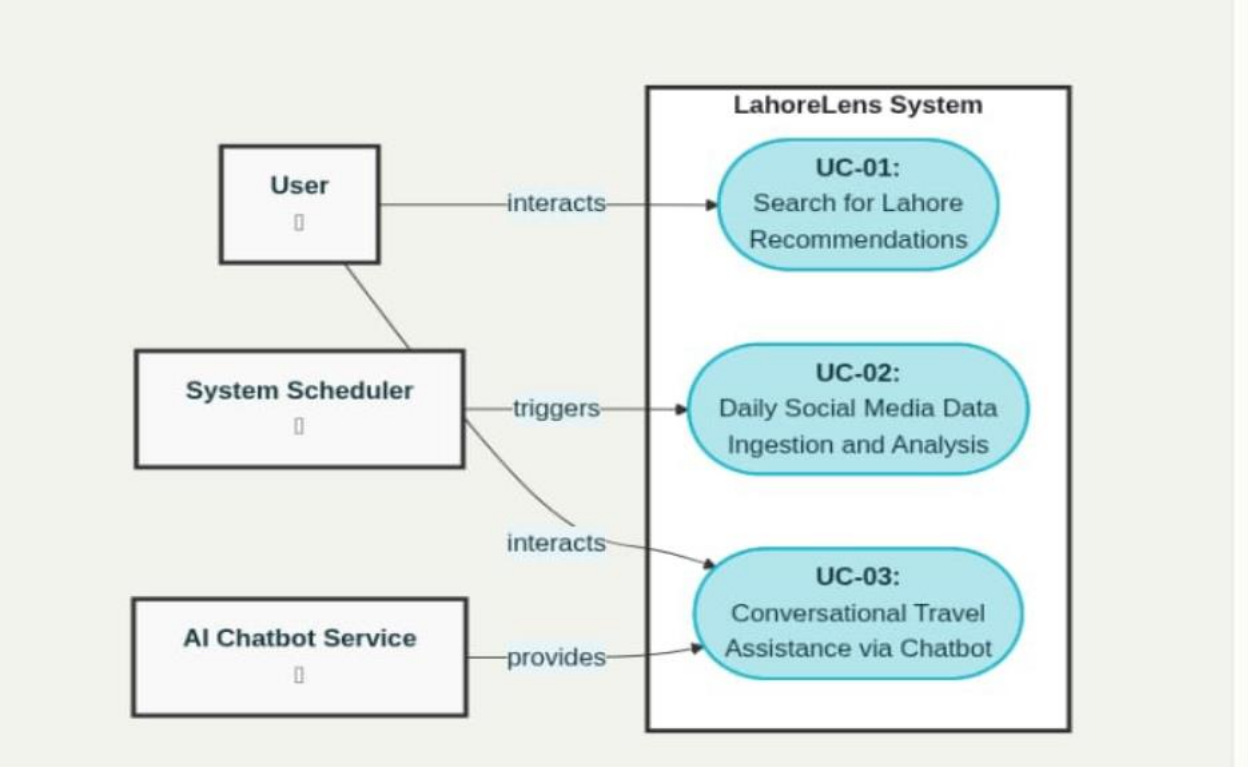


Figure 3.1 LahoreLens Use Case Diagram

This diagram summarizes the core interactions of LahoreLens by showing how the primary actors—User, System Scheduler, and AI Chatbot Service—engage with the system. The User either performs direct searches for Lahore recommendations (UC-01) or interacts conversationally through the chatbot (UC-03), while the System Scheduler periodically triggers the daily social media data ingestion and analysis process (UC-02) that keeps the recommendation knowledge base fresh and trend-aware.

3.2 Activity Diagram

The key activity flow sequence starts as a user opens the web application LahoreLens and enters a search or chatbot query after authenticating as a user. The key activity of the system validates the search or chatbot query by acquiring or updating processed social media insights. The system generates ranked recommendations incorporating locations and sentiment.

Key activities:

• User login or guest access initialization.

• Submission of query (Search Bar/Chat Bot).

• Backend request validation and logging of interactions.

• Retrieval and, where required, update of NLP-processed data from MongoDB.

• Generating recommendations and responses for the React.js front end.

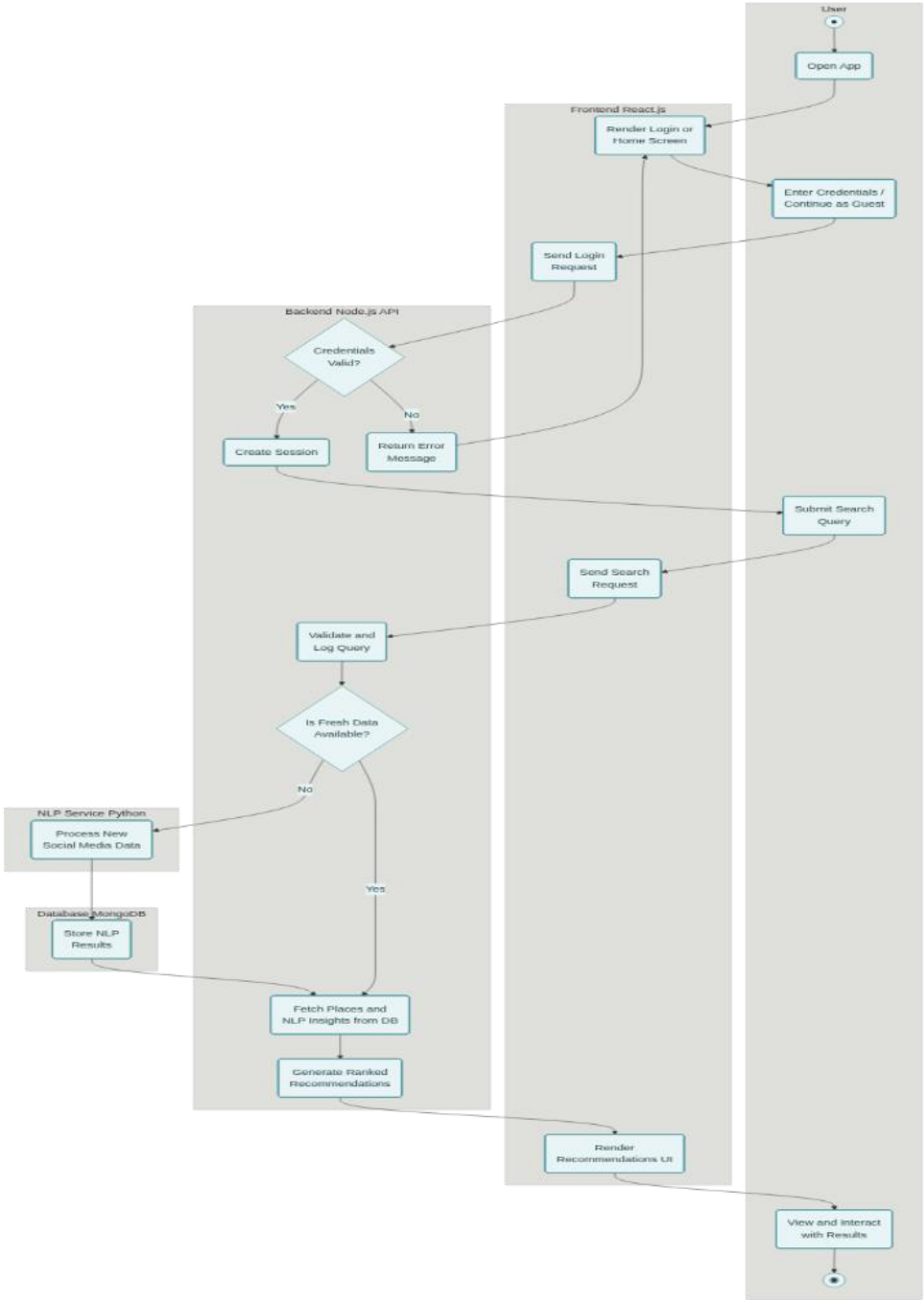


Figure 3.2 Main activity flow of LahoreLens

From opening the application to viewing personalized recommendations, showing interactions across User, Frontend, Backend, NLP Service, and Database.

3.3 Sequence Diagram

The sequence diagram captures the message flow for a typical search operation: User → React Frontend → Node.js Backend → MongoDB → NLP Service (when needed) → back to Frontend.

Typical Sequence Steps:

1. User enters a search query and clicks the "Search" button from the React interface.
2. Frontend makes a REST call to the Node.js Application Service
3. Backend queries MongoDB for already processed entities, sentiments, and topics corresponding to the search query.
4. If the recent data is not available, Backend will request the Python-based NLP service to process the new posts and the database will be updated accordingly.
5. The similarity score and sentiment score trends calculated using Social Network Analysis SNA algorithms within the Backend/Platform.
6. It returns a ranked list of locations to the frontend.
7. Frontend handles the rendering of the list of recommendations and the optional map view.

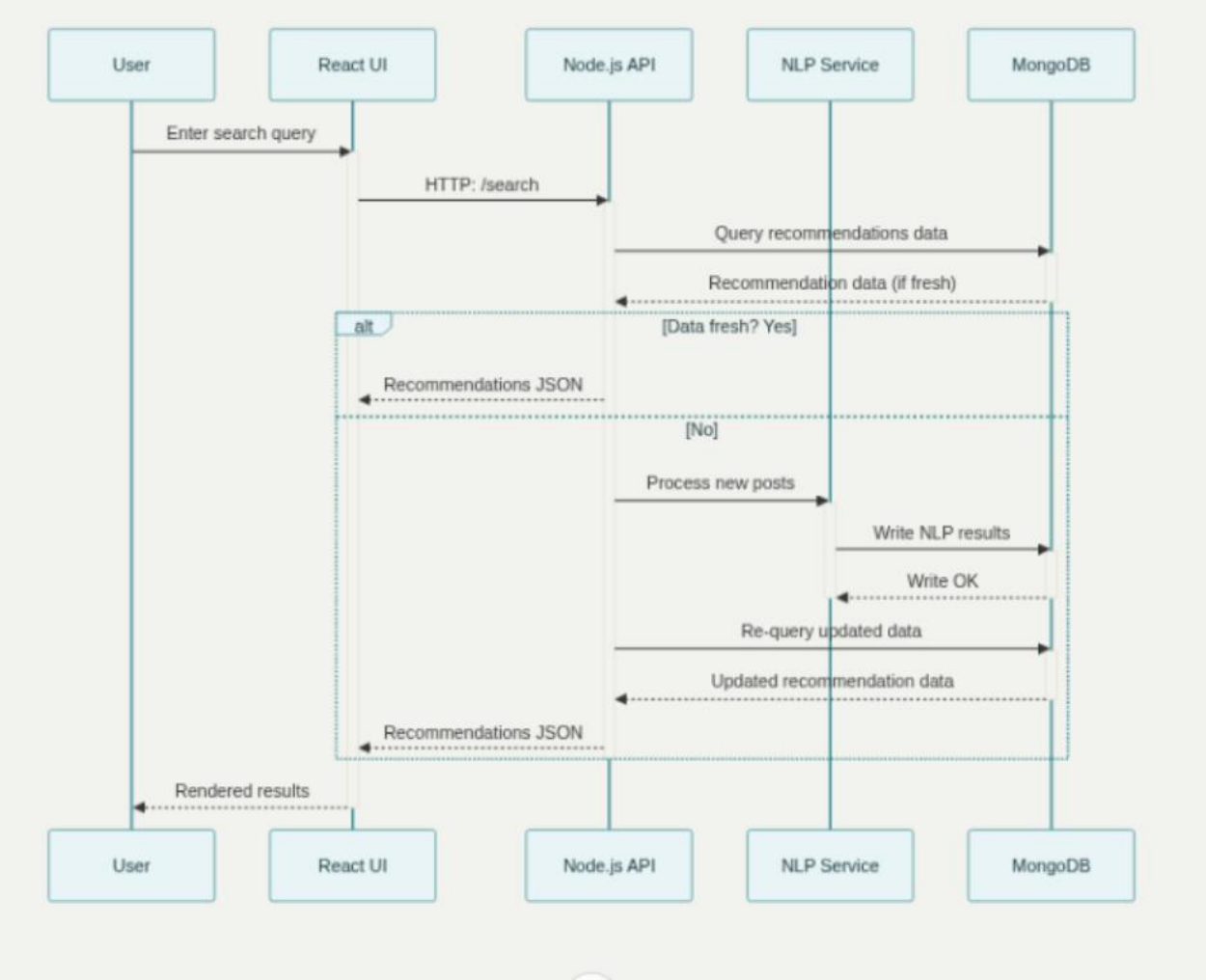


Figure 3.3 Sequence diagram

Figure 3.3 demonstrates flow for a LahoreLens search request showing interactions between the User, React UI, Node.js API, NLP Service, and MongoDB, including an alternative path for refreshing recommendations when data is not fresh.

3.4 Software Architecture Diagram

LahoreLens applies a Microservices Architecture pattern that segregates data-intensive components like AI into a completely separate domain from other user-service-oriented tasks.

- Data Collection Service (Python + APIs/scrapers) for extracting social media posts and comments.
- NLP Processing Service (Programming in Python using NLTK, spaCy, and/or the Transform Toolkit) including Named Entity Recognition, Sentiment Analysis, and topic analysis using Latent Dirichlet Allocation.
- Application Service Backend (Node.js + Express) for implementing REST APIs, Authentication Mechanism, Business Logic, and Social Network Analysis SNA-based Recommendations.
- Presentation Tier Frontend (React.js) for user interface and visualizations.
- MongoDB: Storage of user data, locations, posts, and results from NLP processes.
- Services like Google Dialogflow for chat bot functionality, Google Map service for geocoding, and image APIs for text-to-image mapping.

Figure 3.4 Software architecture of LahoreLens

Figure 3.4 shows the React.js frontend, Node.js backend acting as an API gateway and Facade, Python microservices for data collection and NLP, a central MongoDB database, and external APIs for social media, maps, and chatbot integration.

3.5 Class Diagram

Although MongoDB is a document database, a conceptual class diagram helps to identify entities and relationships. The main conceptual classes are:

- Consumer: "userId, name, email, preferences,
- location: placeId, name, category, coordinates, avg sent., trendscore
- SocialPost: postId, sourcePlatform, rawText, timestamp
- NLPResult : nlpId, entities, sentimentScore
- Recommendation: recommendationId, userId, placeId, similarityScore

Associations show that Place is a gatherer of SocialPost and NLPResult entries, and User is a gatherer of Recommendation entries.

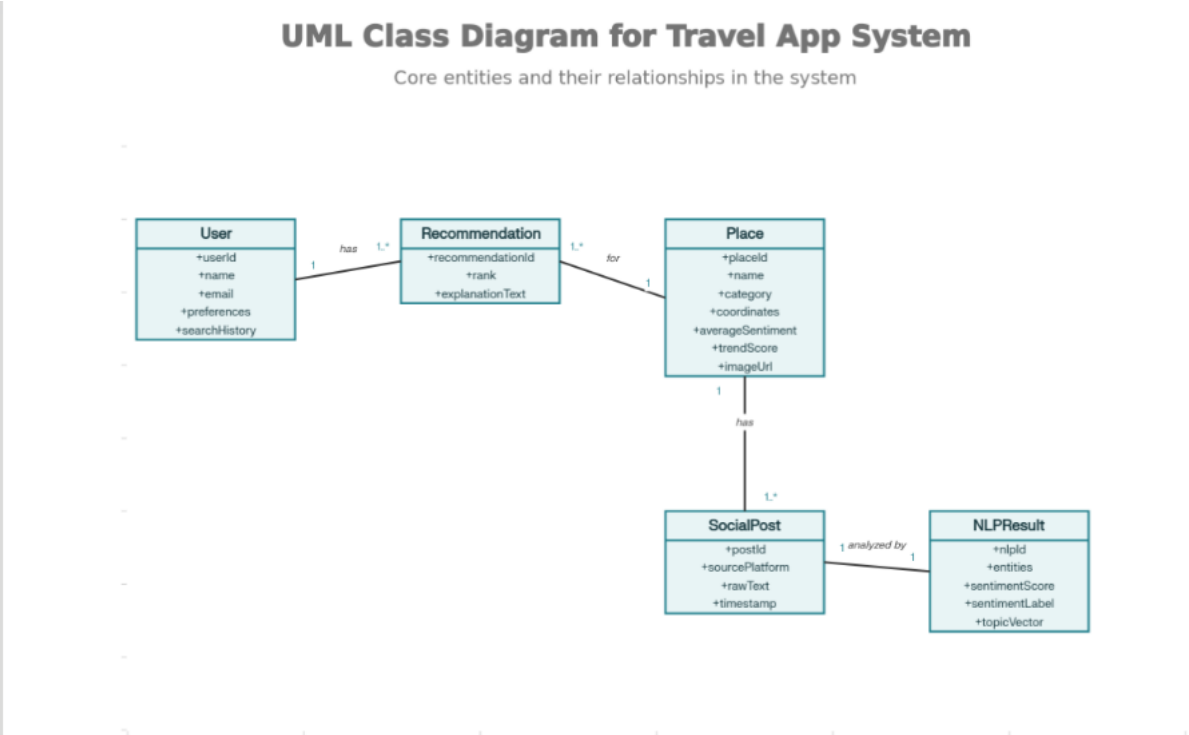


Figure 3.5 Core domain class diagram for LahoreLens

It shows User, Place, SocialPost, NLPResult, and Recommendation entities with their one-to-many and one-to-one relationships.

3.6 Database Diagram

The physical data model in MongoDB includes a variety of data model concepts. The data model has a number of entities. An entity in

Primary Sources:

- users: stores authentication IDs, profile information, and high-level preferences.
- places: stores Lahore locations enriched with metadata from social media and Google Maps (name, coordinates, categories).
- posts: holds the raw and unprocessed text from the social media sources along with the links to the places and sources.
- nlp_results: Stores the entity tags, sentiment values, and topic probabilities.
- interactions/search_logs: tracks user search queries, clicks, and saved locations for analysis and evaluation using SNA.

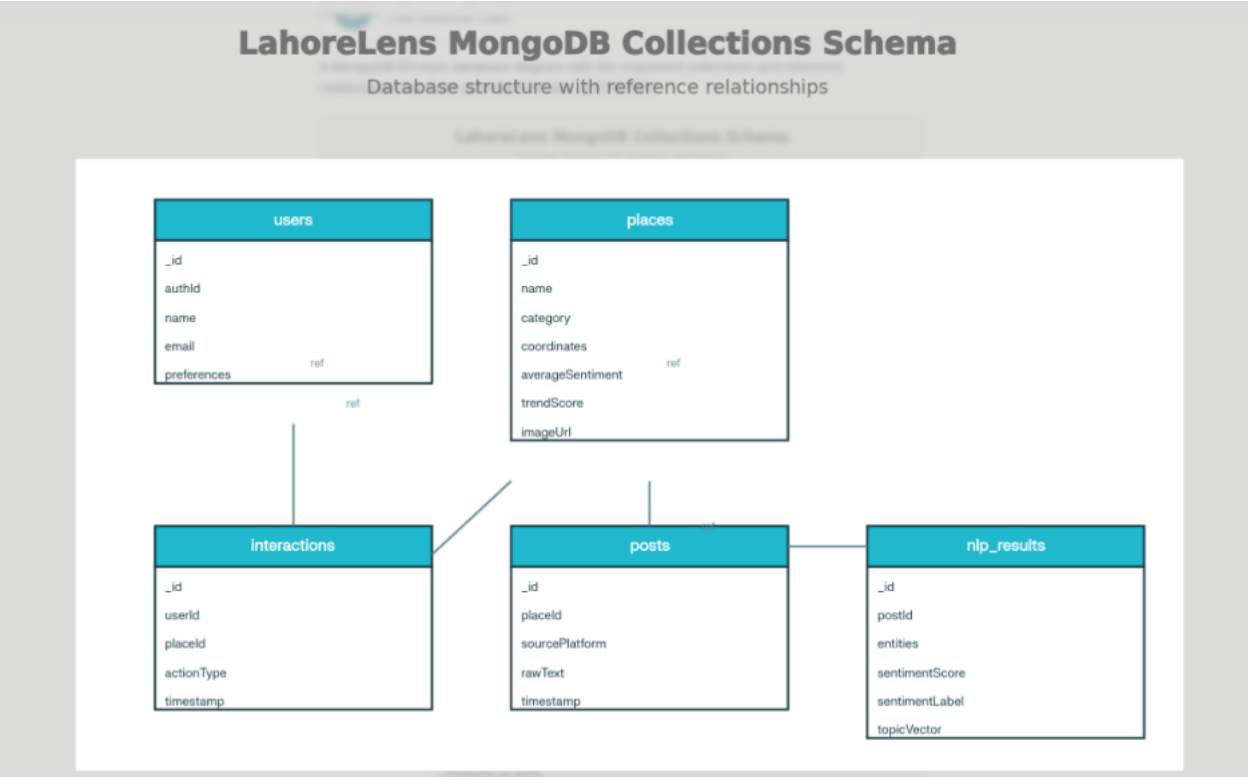


Figure 3.6 MongoDB collection diagram for LahoreLens

It shows users, places, posts, nlp_results, and interactions collections with reference links such as posts.placeId → places._id and nlp_results.postId → posts._id.

3.7 Collaboration Diagram

The collaboration diagram depicts how objects work together to provide a response to a recommendation query. Common objects that work together in such a case include FrontendController, AuthManager, RecommendationEngine, NLPServiceClient, DataCollectionClient,

Message flow:

- FrontendController invokes AuthManager to verify user.
- FrontendController calls Recommendation Engine with the query and user profile.
- Recommendation Engine uses DatabaseAdapter to fetch candidate places and past interactions.
- If freshness thresholds are not met, Recommendation Engine calls NLPServiceClient / DataCollectionClient to trigger analysis of new data.
- Recommendation Engine returns a final ranked list to Frontend Controller for rendering.

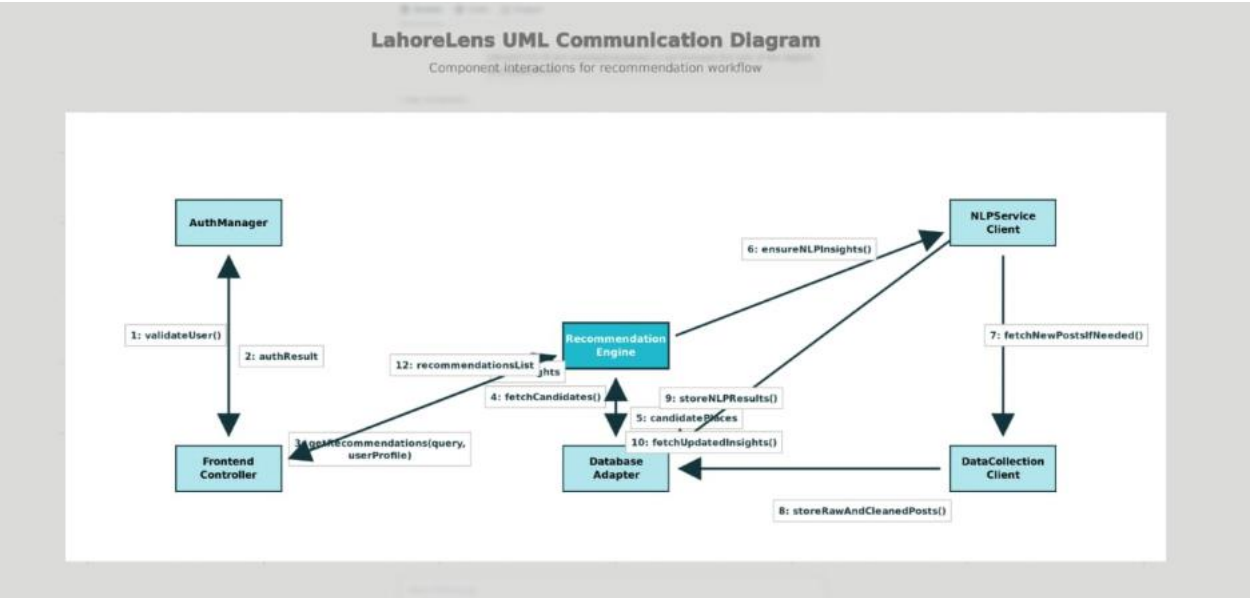


Figure 3.7 Collaboration Diagram

Chapter 4. System Testing

System testing for LahoreLens is organized into three levels: test case design, unit testing, integration testing, and user acceptance testing. For Phase-1, the emphasis is on planned strategy and representative test cases rather than exhaustive execution.

4.1 Test Cases Design

Test cases include definitions for web functionalities, microservices, and AI model assessment. In addition, they include IDs, purposes, inputs, processes, or step-by-steps, and result definitions.

Representative functional test cases:

TC-1: User Login

- Test: Validate that users are able to sign in with proper login credentials using Firebase Authentication.
- Steps: Open application, proceed to login page, fill in valid email and password, submit.
- Actual Result: User authenticated, redirected to dashboard, and session token secured.

TC-02: Search Recommendations

- Test purpose: Verify that when there is data, there are at least three recommendations that match a search term.
- Steps: Enter “street food near Lakshmi Chowk” in search bar and submit.
- Anticipated Output: The ranked list of places with name, type, sentiment summary, and distance is shown.

TC-03: Chatbot Common Query

- Objective: The chatbot should answer a predefined question contained in the FAQ, like ‘tourist spots near Minar-e-Pakistan’.
- Steps: Open chatbot widget, write a question, send.
- Expected Result: At least three recommended sites are returned by the chatbot with corresponding descriptions.

Representative AI model test cases:

TC-NLP-01:

- Objective: To evaluate the performance of Named Entity Recognition tasks on the labeled tourism data in the city of Lahore.
- Expected Result:

  F1-score $\geq$ 0.70 for correct place and event name recognition.

TC-NLP-02:

- Objective:Evaluate the accuracy of the sentiment classification task on a well-balanced testing dataset comprising positive, negative, and neutral comments.
- Expected Result: Overall accuracy rate should be greater than or equal to 0.70; and confusion matrix will be printed for further optimization.
- Precision, recall, F1 value, and accuracy scores will be provided on AI models, whereas pass/fail information will be retained for functional tests.

Table 4.1: System Test Cases and Validation Criteria

ID	Description	Input	Expected Output	Actual Result	Status
TC-01	User login with valid credentials	Email: valid user; Password: correct	User is authenticated, redirected to dashboard, and a session is created with no error message.	As observed	Pass / Fail
TC-02	User login with invalid password	Email: existing user; Password: incorrect	Login is rejected, an error message is shown, and no session is created; user remains on login page.	As observed	Pass / Fail
TC-03	Search for Lahore recommendations	Search query (e.g., “best street food near Lakshmi Chowk”)	At least three recommendations are returned, each with name, category, sentiment summary, and distance.	As observed	Pass / Fail
TC-04	Chatbot response for common query	Chat message (e.g., “tourist spots near Minar-e-Pakistan”)	Chatbot returns a natural-language reply plus at least three relevant suggested places.	As observed	Pass / Fail
TC-05	NER model accuracy on labeled test set	Labeled comments containing Lahore place/event names	Model correctly detects at least 70% of place/event entities; precision, recall, and F1 are recorded.	As measured	Pass / Fail
TC-06	Sentiment classification performance	Balanced test set of positive, negative, and neutral comments	Overall sentiment accuracy is at least 70%; confusion matrix is generated for analysis.	As measured	Pass / Fail
TC-07	Daily data ingestion and NLP processing	Scheduled trigger for data collection pipeline	New posts are fetched, cleaned, analyzed, and stored in MongoDB with no critical failures.	As observed	Pass / Fail
TC-08	Recommendation generation using stored insights	Valid user session and search query with existing processed data	Ranked recommendations are generated in acceptable time (e.g., under 3 seconds) and displayed to the user.	As observed	Pass / Fail

4.2 Unit / Integration / Acceptance Testing

Unit Testing
Unit tests validate individual modules in isolation to ensure correctness and robustness. Examples include:

- Node.js backend controllers and services: input validation, error handling, and response formatting tested with frameworks such as Jest or Mocha.
- Python NLP functions: preprocessing, entity extraction, and sentiment prediction tested with pytest using small, hand-crafted datasets.
- React components: basic rendering and state transitions tested using component testing libraries.

Integration Testing
Integration tests verify that microservices and shared resources interact correctly.

Key integration scenarios:

- Backend ↔ MongoDB: verify CRUD operations for users, places, posts, and recommendations.
- Backend ↔ NLP Service: send a batch of posts, receive processed entities and sentiment, and confirm correct persistence.
- Backend ↔ Dialogflow: send text message and verify recognized intent and parameters are handled correctly.

Mock servers or sandbox environments will be used to simulate external APIs such as Twitter, YouTube, and Google Maps when needed.

Acceptance Testing
User Acceptance Testing UAT evaluates whether the system satisfies user expectations and high-level success criteria. A small group of at least ten target users will be asked to perform typical tasks such as searching for places, using the chatbot, and interpreting recommendation explanations.

Acceptance criteria:

- Core features (login, search, recommendation display, chatbot) operate without critical failures.
- Users are able to discover new, relevant locations in Lahore that they were not aware of previously.
- Average satisfaction score is at least 3 out of 5 on a short questionnaire covering usability, usefulness, and clarity of information.

Flowchart:

```
        +----------+
        |  Start   |
        +----+-----+
             |
             v
+-------------------------+
|    Define Test Cases    |
+-----------+-------------+
            |
            v
+-------------------------+
|   Perform Unit Testing  |
+-----------+-------------+
            |
            v
+----------------------------+
| Perform Integration Testing|
+-------------+--------------+
              |
              v
+----------------------------+
|   Run AI Model Evaluation  |
+-------------+--------------+
              |
              v
+-------------------------------------------+
| Conduct User Acceptance Testing (UAT)     |
+--------------------+----------------------+
                     |
                     v
+-------------------------------------------+
|     Record Metrics and Defects            |
+--------------------+----------------------+
                     |
                     v
+-------------------------------------------+
|        Refine Models and Code             |
+--------------------+----------------------+
                     |
                     v
              +------+------+
              |     End     |
              +-------------+
```

Figure 4.1 Process Flowchart

Chapter 5. Implementation

This section provides the implementation details of the project. For the FYP Mid-Year Report, these details comprise **30% implementation** of the project, focusing on the system architecture setup, data collection pipeline, and the initial frontend prototype.

5.1 Work Breakdown Structure (WBS)

A Work Breakdown Structure (WBS) is a hierarchical decomposition of the project into smaller, manageable tasks. The project is divided into four main phases to ensure a systematic approach to Research and Development. For this Mid-Year Report (FYP-I), we have completed Phase 1 (Planning & Research) and Phase 2 (Data Engineering & Prototyping).

Phase 1: Research & Environment Setup (Completed)

- Conducted comprehensive literature review and comparative analysis.
- Setup of development environment (Node.js, Python, MongoDB, React).
- Selection of AI libraries (spaCy, NLTK, Scikit-learn).

Phase 2: Data Engineering & Prototyping (Completed - 30% Milestone)

- Development of custom web scrapers for social media data (Twitter, Reddit, YouTube).
- Creation of the initial raw dataset (~30,000+ comments).
- Development of the React.js Frontend Interface (Home, Search, and Map views).

Phase 3: AI Model Development (Planned for FYP-II)

- Implementation of the NLP Pipeline (NER, Sentiment Analysis).
- Development of the LDA Topic Modeling engine.
- Integration of the Social Network Analysis (SNA) module.

Phase 4: System Integration & Evaluation (Planned for FYP-II)

- Integration of AI microservices with the Node.js backend.
- End-to-end testing and user acceptance testing.
- Final deployment to cloud infrastructure (AWS).

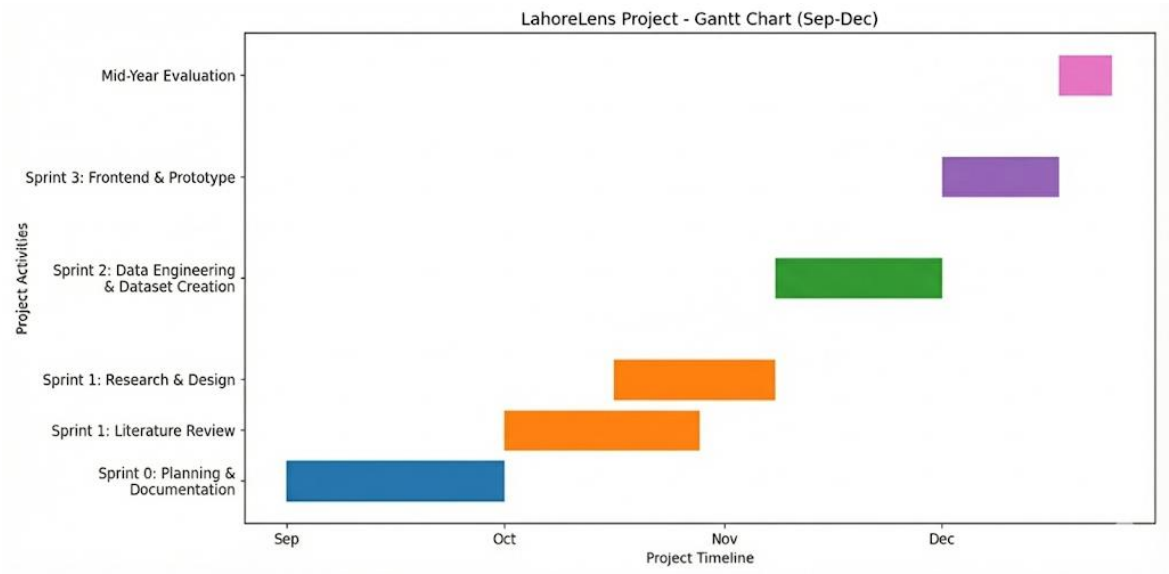


Figure 5.1 Phase 1 Timeline

Table 5.1: Phase 1 Activity Log (Sept - Dec 2025)

Month	Sprint / Phase	Key Activities Completed	Status
September 2025	**Sprint 0: Planning**	• Finalized topic "LahoreLens" & submitted proposal. • Selected MERN Stack + Python Architecture. • Defined project scope and project details.	**Completed**
October 2025	**Sprint 1: Research**	• Conducted Requirements Elicitation (Section 2.3). • Designed Use Case & System Diagrams. • Set up GitHub repository and development environment.	**Completed**
November 2025	**Sprint 2: Data Engineering**	• **Developed Custom Scrapers:** Built Python scripts for X (Twitter), Reddit, Facebook, and YouTube. • **Dataset Creation:** Successfully aggregated & cleaned 30,000+ social media comments. • **API Integration:** Connected to initial public APIs.	**Completed**
December 2025	**Sprint 3: Frontend & Prototype**	• **Frontend Development:** Built nearly complete React.js UI (Home, Search, Map views). • **Backend Setup:** Initialized Node.js server structure. • **Integration Testing:** Verified connection between Scrapers and Database. • **Mid-Year Progress Evaluation.**	**Completed**

5.2 Team Roles and Responsibilities

The project workload is distributed among the team members to ensure efficient execution of the diverse R&D components. The roles are divided between the **Development Team** (Frontend/Backend) and the **Research Team** (Data/AI). This project requires a significant one-year effort due to the complexity of building a custom NLP pipeline and processing unstructured social media data. Unlike standard CRUD applications, this study involves iterative training of AI models and extensive data cleaning, justifying the collaborative effort of the group.

Table 5.2. Individual Tasks

Team Member	**Activity**
M. Arham Nadeem (Group Leader)	**Activity 01:** Backend Architecture & Setup
	Activity 02: Data Scraping Script Development
	Activity 03: Project Management & Documentation Lead
Ayesha Yaqoob	**Activity 01:** Data Preprocessing & Cleaning
	Activity 02: Literature Review & Research
	Activity 03: Requirement Analysis & SRS
Maaz Mughal	**Activity 01:** Social Media Data Collection
	Activity 02: AI Model Feasibility Research
	Activity 03: Dataset Validation & Quality Check
Kashif Ali	**Activity 01:** UI/UX Design & Wireframing
	Activity 02: Frontend Development (React.js)
	Activity 03: API & Map Integration

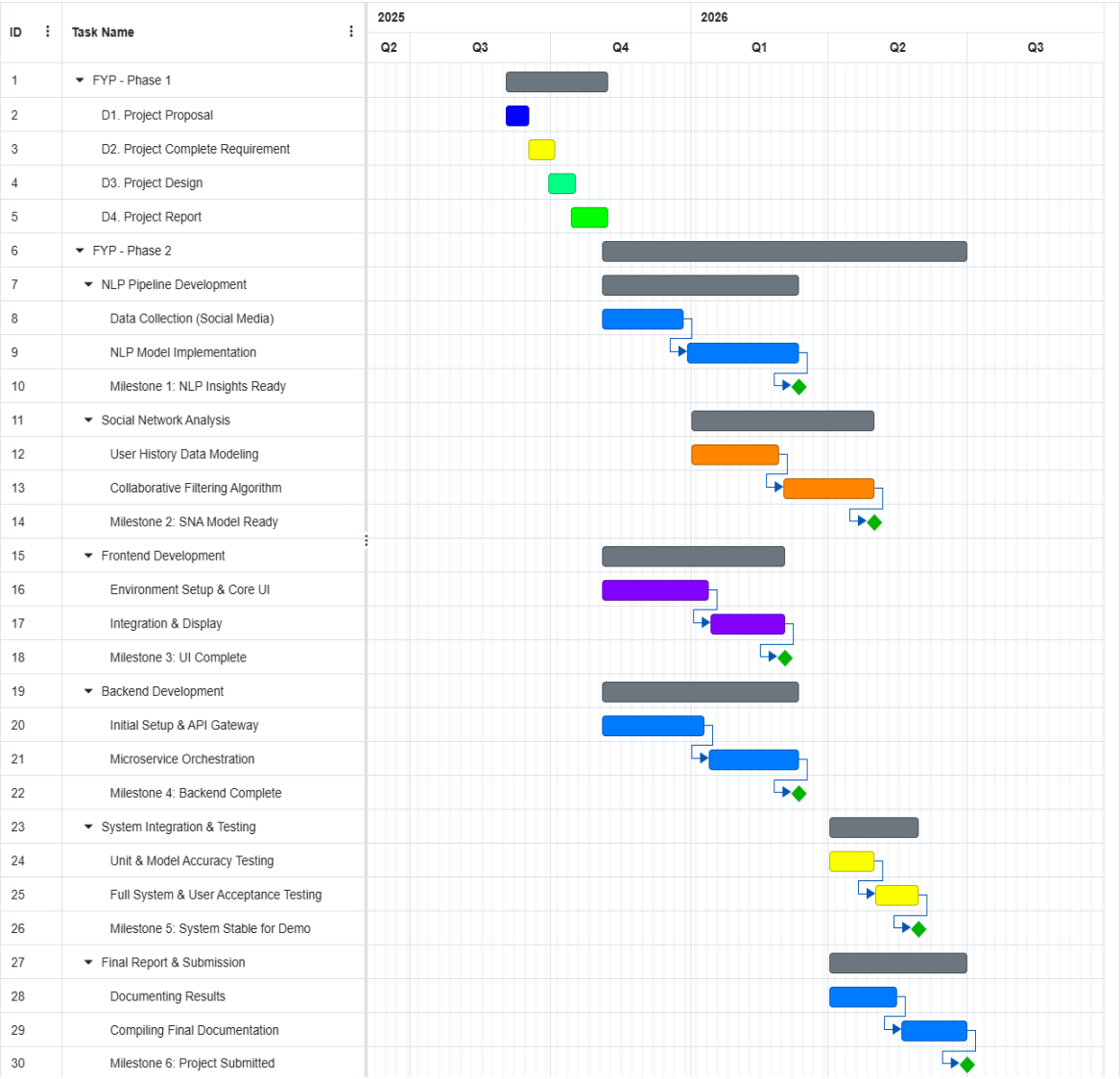


Figure 5.2 Gannt Chart

5.3 Tools and Technologies

The following software tools, frameworks, and platforms have been selected to build the LahoreLens system. These choices ensure that the project relies on industry-standard, scalable, and modern technologies.

- **Python:** The primary programming language selected for all Data Engineering and AI tasks. Its extensive ecosystem (Pandas, NLTK) makes it ideal for processing unstructured text.
- **React.js:** Selected for the Frontend interface. React's component-based architecture allows us to build a dynamic, single-page application (SPA) that provides a smooth user experience.
- **Node.js & Express.js:** Used for the Backend Application Service. Node.js handles the API requests and acts as the orchestrator between the Frontend and the Python AI microservices.
- **MongoDB:** A NoSQL database chosen for its flexibility. It stores the unstructured social media data (JSON format) and the processed insights without requiring a rigid schema.
- **Selenium & BeautifulSoup:** Python libraries used for the Data Collection Pipeline. These tools allow us to scrape public comment histories from web platforms that lack direct APIs.

5.4 Implementation Details

To realize the proposed solution, we have implemented the initial phase of the system, focusing on data acquisition and the user interface. The implementation process was divided into three core modules: Environment Configuration, Data Collection Pipeline, and Frontend Prototyping.

1. Design Choices and Architecture

We chose a Microservices Architecture to separate the computationally heavy AI tasks (Python) from the lightweight user interaction logic (Node.js). This ensures that the data processing pipeline does not block the user interface.

2. Step-by-Step Implementation Process

Step 1: Environment Setup & Configuration

We established a robust development environment using Git for version control. We initialized the React.js project using create-react-app and set up the Node.js backend with Express. For the Python services, we created virtual environments to manage dependencies like pandas and selenium.

Step 2: Data Collection Pipeline

A major R&D challenge was acquiring a local dataset for Lahore.

- **Scraper Development:** We wrote custom Python scripts using Selenium to automate the extraction of comments from public Reddit threads (e.g., r/Lahore) and YouTube travel vlogs.
- **Data Aggregation:** We successfully aggregated over 3,000 raw comments.
- **Cleaning:** We implemented a basic cleaning script to remove emojis, URLs, and duplicate entries, resulting in a "Gold Standard" dataset ready for Phase 2 training.

Step 3: Frontend Development

We developed the core user interface to demonstrate the application flow.

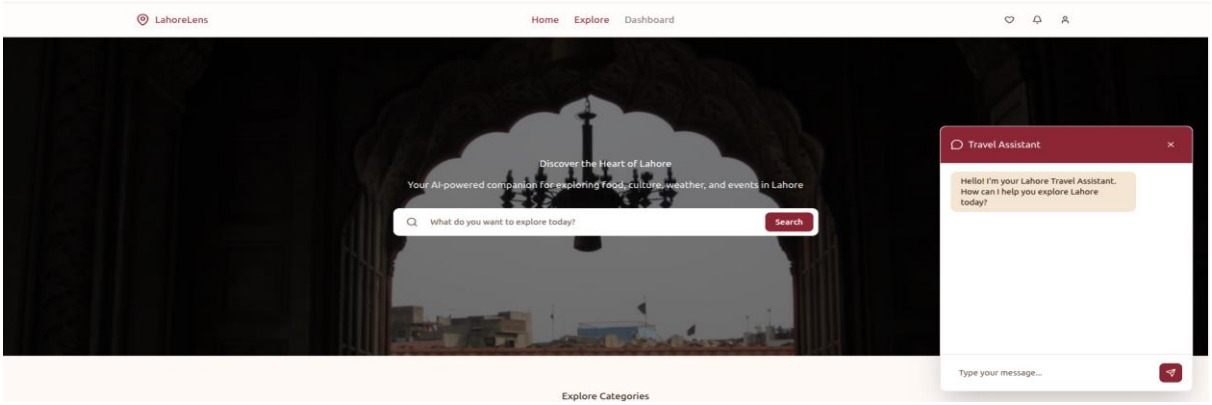
- **Home Page:** Implemented a search bar and a "Trending Now" section.
- **Responsiveness:** Ensured the layout adapts to running desktop.

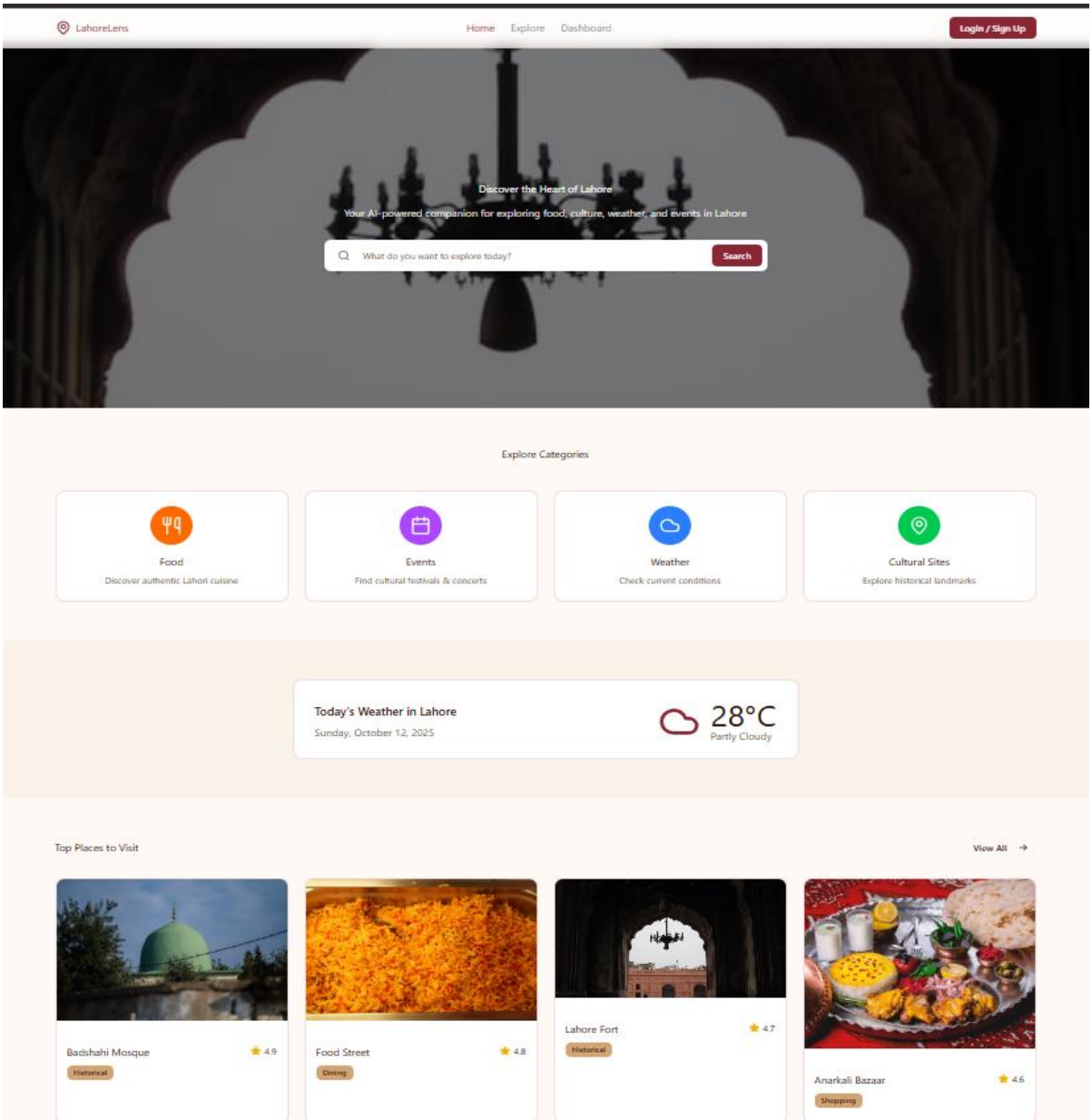
3. Prototype Completion

The frontend prototype is set to be fully functional as a standalone interface, and is scheduled for final completion by the end of December 2025. It is designed and ready for integration with the backend API in the subsequent development phase (FYP-II).

5.5 Screenshots of Prototype / System

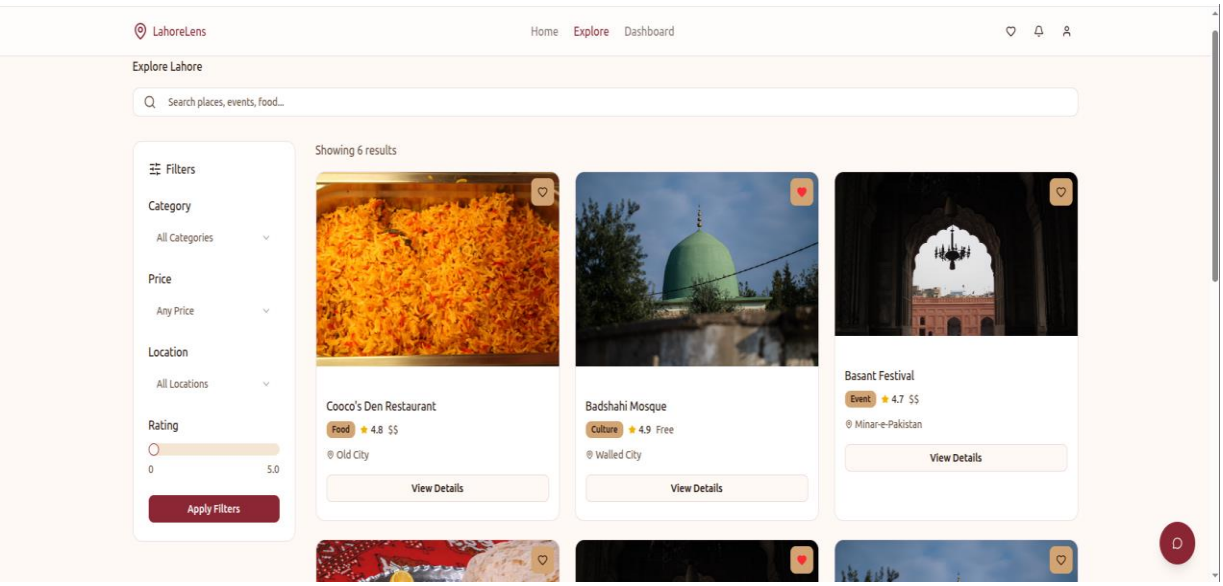
1. The Application Homepage





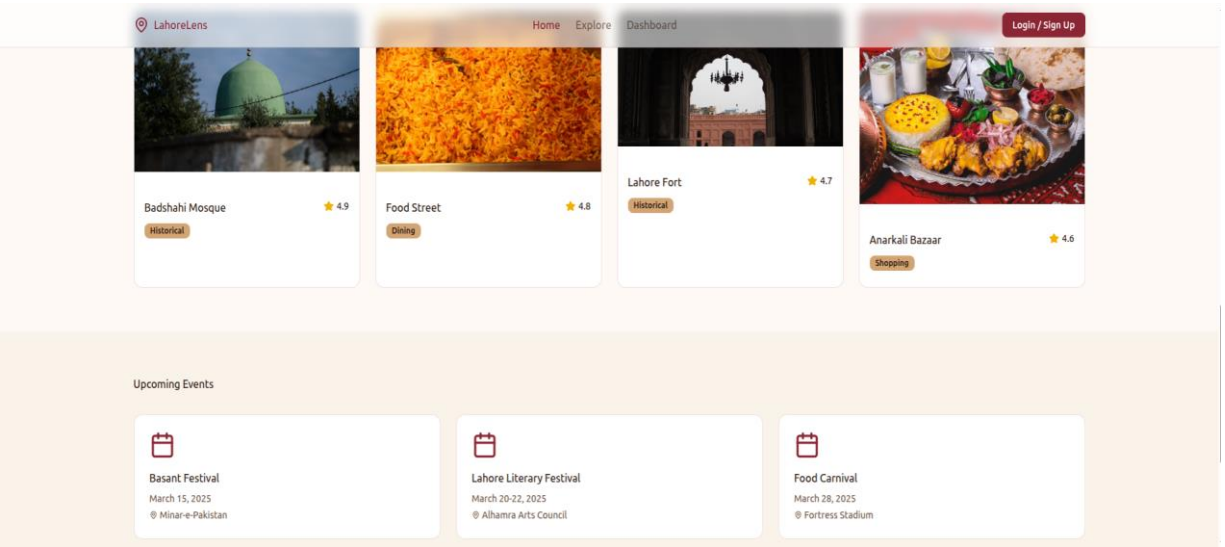
Description: Figure 5.1 displays the "LahoreLens" homepage. It features a clean, search-centric interface designed to guide users immediately to their interests. The "Trending" section at the bottom is a placeholder that will eventually display real-time recommendations generated by the backend.

2. The Search Interface



Description: Figure 5.2 illustrates the Search Results page. On the left, a list of venues is displayed.

3. The Dynamic Events Dashboard



Description: Figure 5.3 illustrates the "Upcoming Events" module within the application homepage. This interface is designed to highlight culturally significant activities (e.g., Basant Festival, Lahore Literary Festival).

5.6 Challenges During Implementation

During the initial 30% implementation phase, the research team encountered several technical hurdles related to data acquisition and processing. The following points summarize these challenges and the engineering solutions employed to resolve them.

1. API Rate Limiting & Access Restrictions

- **Challenge:** Integrating the **X (Twitter) API** and **Reddit API** presented significant hurdles due to strict rate limits (HTTP 429 errors) and restricted access to historical data on free-tier plans. This threatened to bottleneck our data collection pipeline, leaving us with insufficient training data for the NLP models.

- **Solution:** We implemented a hybrid acquisition strategy. For the Reddit API (PRAW), we added error handling to automatically pause requests when rate limits are hit (exponential backoff). For X (Twitter), where API access is highly restricted, we developed custom Selenium scripts to simulate user browsing and scrape public timelines, effectively bypassing standard API volume limitations.

2. Parsing Dynamic Web Content

- **Challenge:** Platforms like YouTube and Facebook utilize Single Page Application (SPA) frameworks, loading comments asynchronously via AJAX/JavaScript only when a user scrolls. Standard HTTP request libraries (like Python's requests) failed to capture this dynamically rendered content, returning empty HTML shells.

- **Solution:** We migrated our scraping architecture to **Selenium WebDriver**. This allows us to instantiate a "headless" Chrome browser that executes JavaScript and mimics human scrolling behavior, ensuring the full Document Object Model (DOM) is loaded before data extraction occurs.

3. Linguistic Complexity (Roman Urdu)

- **Challenge:** A significant portion of the collected social media text (approx. 40%) consists of "Roman Urdu" (Urdu written in English script) and code-mixing (e.g., *"Food bohat tasty tha"*). Standard NLP libraries (NLTK, spaCy) treat these non-English terms as noise or spelling errors, degrading the accuracy of our Sentiment Analysis.

- **Solution:** We identified this early and have planned a specialized cleaning pipeline for the upcoming FYP-II. This involves compiling a custom **"Roman Urdu Stop-Word List"** and mapping common transliterated terms to their English sentiment equivalents (e.g., mapping *"bura"* to *"bad"*) to ensure the model correctly interprets local context.

4. Data Noise & Contextual Relevance

- **Challenge:** Raw data streams from r/Lahore and X often contain high volumes of off-topic discussions (e.g., political debates, buy/sell spam) that create "noise" in the dataset. Training the Recommendation Engine on this data would lead to irrelevant suggestions.

- **Solution:** We implemented strict keyword-based filtration logic during the scraping execution. The scripts are configured to only ingest records containing specific tourism domain tokens (e.g., *"restaurant"*, *"visit"*, *"traffic"*, *"view"*, *"ticket"*, *"family"*), automatically discarding approximately 60% of irrelevant noise at the source.

Chapter 6. References

[1] Statista, “Pakistan Travel & Tourism Market Forecast 2025–2029,” 2025.

[2] A. S. K. Xin, H. Y. Ting and A. F. Atanda, “Trends In Tourism Recommendation Systems: A Review,” *Journal of Computing Research and Innovation,* vol. 9, no. 2, pp. 85-107, 2024.

[3] C. Li, X.-Y. Li and G.-P. Li, “A proactive tourism recommender system: Incorporating emotion-based preferences from online reviews,” *Information Processing & Management,* vol. 61, no. 5, p. 105021, 2024.

[4] S. B. K. Kumar, L. K. Sahoo and S. J. D. Kumar, “Sentiment Analysis on Twitter Data for Tourism Recommendation Using Hybrid Deep Learning Model,” *IEEE Access,* vol. 10, pp. 60319-60331, 2022.

[5] L. Kzaz, D. Dakhchoune and D. Dahab, “Tourism Recommender Systems: An Overview of Recommendation Approaches,” *International Journal of Computer Applications,* vol. 180, no. 20, pp. 9-13, 2018.

[6] T. Berka and M. Plößnig, “Designing Recommender Systems for Tourism,” in *ENTER 2008 PhD Workshop,* Salzburg, 2008.

[7] Y. Liu, C. Wang and F. Sun, “A BERT-based Approach for Named Entity Recognition from Social Media Text for Location-based Services,” in *COLING 2020,* Barcelona, 2020.

[8] D. M. Blei, A. Y. Ng and M. I. Jordan, “Latent Dirichlet Allocation,” vol. 3, p. 993–1022, 2003.

[9] K. Riaz, “Rule-based Named Entity Recognition in Urdu,” in *Proceedings of the 2010 Named Entities Workshop,* Uppsala, 2010.

[10] B. A. Hassan, A. A. Hassan, J. Lu, A. M. Ahmed and T. A. Rashid, “A Proposed Hybrid Recommender System for Tourism Industry in Iraq Using Evolutionary Apriori and K-means Algorithms,” 2025.

Lahore Lens

ORIGINALITY REPORT

7%	4%	2%	5%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

MATCH ALL SOURCES (ONLY SELECTED SOURCE PRINTED)

3%

★ Submitted to Higher Education Commission Pakistan

Student Paper

Exclude quotes	Off	Exclude matches	Off
Exclude bibliography	Off		

Appendix A